Cal Poly Humboldt School of Engineering

Spring 2026 ENGR 326 Computational Methods III

Modeling the effective mass reduction of circuitry given enhanced thermal dissipation

Client: Aubrey Cooper,

EMC Motors, Co founder

Submitted by: Kaden Smith

May 15, 2026



Table of contents

Executive Summary **1**

Introduction **2**

Overview of the problem 2

Objectives 2

Importance of solving the problem 3

Background **3**

Client background 4

Problem situation 4

Relevant literature 5

Method options and justification 6

Methods **7**

Math model 7

Numerical method 9

Application **10**

Math model parameters 11

Boundary conditions 11

Numerical method parameters 12

Code outline 13

Results **14**

Objective 1: Temperature distribution 14

Objective 2: Current vs. temperature response 14

Objective 3: Effective ampacity 15

Objective 4: Thermal interaction effects 17

Summary of results 18

Discussion **19**

Conclusions **24**

Recommendations **25**

Appendix I: Program verification **28**

Appendix II: Program source code **30**

List of figures

1 The modeled 16 cell bus bar geometry defines the cell arrangement, contact areas, output patch, and embedded heat pipe regions used in the model. 5

2 The actual heat pipe bus bar model shows negligible change in final maximum temperature when the time step is varied around the selected baseline value. Percent change in final maximum temperature is shown relative to the baseline time step of $\Delta t = 2.0$ s at 350 A. 13

3 The solid bus bar reaches its highest temperatures near the contact and output regions at 400A. 15

4 The heat pipe bus bar spreads heat more evenly along the bus bar and reduces the local peak temperature at 400A. 15

5 Maximum temperature increases with current for both bus bar configurations. The heat pipe configuration reduces peak temperature at each tested current level. . . . 16

6 Temperature increases over time and approaches steady behavior during the 400A run. 16

7 The solid bus bar current density field at 400A shows current crowding near the output region. 17

8 The heat pipe bus bar current density field at 400A shows how the modified material region changes the electrical loading pattern. 18

9 Joule heating in the solid bus bar is concentrated in regions with higher current density. 18

10 Joule heating in the heat pipe bus bar changes with the embedded high conductivity regions and current path. 19

11 Increasing the cell heat generation rate increases both maximum and average temperature. 21

12 Heat pipe spacing affects peak temperature because placement changes how heat spreads through the bus bar. 22

13 Higher heat pipe axial conductivity reduces peak temperature in the heat pipe bus bar case. 23
I.1 Numerical temperature predictions approach the analytical Newton cooling solution as the time step decreases, verifying the transient thermal update. 29
I.2 Final temperature error decreases as the time step is refined, showing convergence of the implicit Euler verification problem. 30

List of tables

1 Math model parameters for the 16 cell thermal electrical system. 11
2 Numerical method parameters used in the final model runs. 12
3 Actual model convergence results show that the final maximum temperature is insensitive to time step refinement near the selected baseline. 13
4 Final temperature results for solid and heat pipe bus bar configurations. 17
I.1 Verification error decreases as the time step is refined. 29

EXECUTIVE SUMMARY

EMC Motors is a start up electric motorcycle company working to develop sodium ion battery systems that are affordable, durable, and repairable. Aubrey Cooper, Co founder of EMC Motors, identified a key design challenge: the bus bars used to connect the battery cells in the motorcycle power system generate a significant amount of heat under high current. The normal solution is to add more copper, but this increases cost and volume. Increasing the cross sectional area of the conductor also doesn't solve the main issue, localized hot spots near cell terminals leading to decreased cycle life of the system. This project looks at whether improved thermal dissipation can reduce the thermal limits of the bus bar while decreasing the overall material used for the conductor.

This modeled system consists of 16 cylindrical battery cells arranged in two offset rows and connected by a copper bus bar. Two configurations were evaluated: a solid copper bus bar and a modified bus bar with embedded high conductivity regions representing integrated commercial copper heat pipes. Both the bus bar and the battery cells generate heat during operation. This is modeled with the bus bar generating heat through Joule heating, and the cells generate heat through internal resistance.

The model treats temperature as a function of position and time. Electrical current distribution is solved using conservation of charge, and then resulting Joule heating is applied to the thermal model. Thus the heat equation is discretized using a finite difference method on a three dimensional grid. After spatial discretization, the thermal model becomes a large system of ordinary differential equations in time. This type of approach is consistent with finite difference methods for partial differential equations and numerical time integration methods for ordinary differential equations (Chapra and Canale, 2015). The system is advanced using a first order implicit Euler method with LU factorization.

The final current sweep tested in amperage (A) was 300, 350, and 400. Both the solid and heat pipe configurations passed the 300A and 350A cases. At 400A, both configurations stayed below the 80°C local temperature limit, but both exceeded the 60°C average temperature limit. Thus, the tested effective ampacity is at least 350A but below 400A.

Sensitivity analysis showed that cell heat generation, heat pipe spacing, and heat pipe axial thermal conductivity affect the peak temperature. The heat pipe spacing and axial conductivity are useful design variables, while cell heat generation should be treated as a physical input based on the actual sodium ion cells, with the sensitivity analysis representing different cell chemistry. Overall, the model does support continued development of the heat pipe bus bar concept, however the design should be validated with physical testing before it is used for final production sizing.

INTRODUCTION

This section includes Overview of the Problem, Objectives, and the Importance of solving the problem.

Overview of the problem

High direct current (DC) power systems used in electric transportation and energy storage rely on bus bars to move large amounts of current between battery cells and the rest of the system. As current increases, the resistance of the conductor creates heat. This temperature rise limits how much current the bus bar can safely carry. A common solution is to add more copper, but that increases cost, mass, and packaging difficulty.

In battery systems, this issue is more complicated because the cells also generate heat during discharge. The bus bar and cells are physically connected, so heat from one part affects the other. This can create local hot spots near cell contacts, even when the average temperature of the system looks acceptable. Because of this, the model needs to look at the temperature distribution through the bus bar and cells, not just a single average temperature.

Objectives

The objectives of this report are as follows:

1. Develop a three dimensional model that predicts the temperature distribution in a copper bus bar and nearby battery cells.

2. Compare how current changes the maximum and average temperature of the system.

3. Determine the effective ampacity of the bus bar system using maximum and average temperature limits.

4. Compare a solid copper bus bar with a bus bar that uses embedded high conductivity regions to represent heat pipe features.

Importance of solving the problem

Improving the thermal performance of high current electrical systems is important because conductors can become heavy and expensive when they are sized only by adding more material. In an electric motorcycle, this matters because the battery area has limited space, and extra copper adds cost and mass. If heat can be spread more effectively, the same electrical demand may be met with less thermal risk.

Prior work has shown that bus bar ampacity depends strongly on temperature rise, material properties, and boundary conditions (Coneybeer et al., 1994). Heat pipe based thermal management has also been shown to reduce local hot spots by spreading heat away from high temperature regions (Smith et al., 2018; Yang et al., 2025). This project applies those ideas to a simplified 16 cell battery and bus bar layout.

This report is organized as follows: Information about the client, problem, and method options are presented in the Background section. The math model and numerical approach are presented in the Methods section, and implementation details are provided in the Application section. The Results and Discussion sections present and interpret the model findings. The findings are summarized in the Conclusions section, and recommended next steps are provided in the Recommendations section.

BACKGROUND

This section includes the Client background, Problem situation, Relevant literature, and Method options and justification.

Client background

EMC Motors is a startup electric motorcycle company focused on improving the availability of environmentally friendly transportation. The company is currently pursuing sodium ion battery systems that are affordable, durable, and repairable. For an electric motorcycle, the battery and power delivery systems are especially important because available space is limited.

Aubrey Cooper, co founder of EMC Motors, is the client for this project. The problem is not just whether a bus bar can carry current, but whether it can carry the necessary current demanded by the drivetrain without exceeding the battery systems maximum operating temperature. In this system, increasing the cross sectional area of the copper bus bar can reduce resistance and temperature rise, but it also increases cost, weight, and fitment difficulty. Thus, this project focuses on whether improved thermal dissipation can allow a bus bar to meet the same electrical demand while reducing the amount of copper needed.

Problem situation

Battery systems require conductors that can move electrical current between battery cells, modules, and electronics. In this project, the conductor is a copper bus bar connected to a set of cylindrical battery cells. As current is drawn from the battery pack, the electrical resistance of the bus bar and the internal resistance of the cells generate heat. Thus, the final temperature is representative of both bus bar heating and cell heating.

The main design issue is that the hottest spots in the system may not be obvious from electrical calculations alone. A bus bar's average temperature can be considered acceptable while still creating local hot spots near cell contacts or high current density regions. This matters because the effective ampacity of the system is limited by the maximum local temperature and the average system temperature.

The physical system is simplified as a 16 cell pack connected by a copper bus bar. Two configurations are compared: a solid copper bus bar and a modified bus bar with embedded heat pipe regions running along the length of the bus bar. The heat pipes are not modeled as full two phase fluid systems in this project. Instead, they are represented as high conductivity regions that

spread heat more effectively than solid copper. This keeps the model realistic enough to compare design trends while still being simple enough to complete within the project scope.

A schematic of the modeled battery and bus bar layout is included to define the geometry used in the analysis (Figure 1).

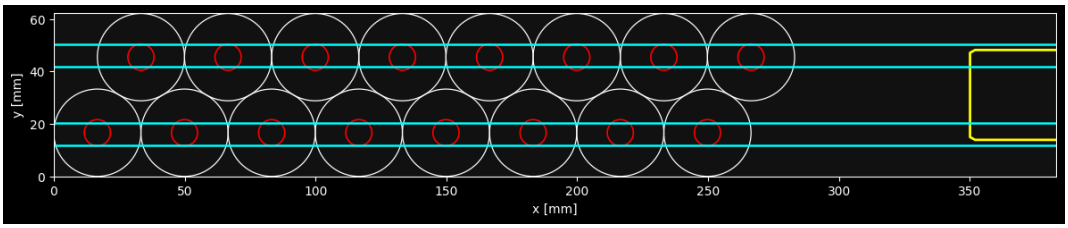


Figure 1. The modeled 16 cell bus bar geometry defines the cell arrangement, contact areas, output patch, and embedded heat pipe regions used in the model.

Relevant literature

Prior work using thermal modeling shows that the current carrying capacity of a bus bar is strongly connected to temperature rise, material properties, and boundary conditions. Bus bars are often sized using electrical resistance and allowable temperature limits, but this can become conservative when the full geometry is not modeled. More detailed thermal models are useful because they show where heat is generated, where it spreads, and where hot spots are most likely to occur (Coneybeer et al., 1994).

Existing literature on battery thermal management also supports the need to model the cells and conductor together. Cylindrical cells do not behave like perfect heat sources. Since battery cells generate internal heat during discharge, the bus bar and cell pack affect each other. This means the bus bar design cannot be evaluated by only looking at the conductor itself (Smith et al., 2018).

Heat pipes are commonly used when heat needs to be moved away from a local region faster than the thermal properties of solid metal would allow. In electronics and battery systems, heat pipe cooling can reduce peak temperature by spreading heat over a larger area before it is removed by convection or another heat sink. For this project, that idea is applied to a bus bar by comparing a solid copper design against a design with embedded high conductivity regions (Yang et al., 2025).

The literature supports this project's main assumption: a bus bar should be evaluated as a coupled thermal electrical system, not as a single piece of copper with a current rating. Prior studies provide the physical basis for Joule heating, cell heat generation, and thermal spreading, while this project applies those ideas to a simplified battery layout. Thus, the goal is not to perfectly reproduce every detail of a commercial heat pipe, but to determine whether enhanced thermal dissipation could reduce peak temperature and improve the usable current range of the bus bar.

Method options and justification

This subsection outlines the available and chosen methods used for the math model and numerical method.

Math model

Several levels of modeling could be used for this problem. The simplest option would be a lumped thermal model, where the bus bar and cells are each treated as having one average temperature. This would be fast to solve, but it would not show local hot spots or the temperature difference between the bus bar centerline and the cell contact regions. Since local temperature is one of the main limits for ampacity, a lumped model would remove too much of the behavior that this project is trying to study.

A more detailed model would be a full computational fluid dynamics and two phase heat pipe model. That type of model could include evaporation, condensation, wick behavior, and fluid flow inside the heat pipe. However, this would require significantly more parameters and would be too large for the scope of this project. It would also make it harder to separate the bus bar thermal behavior from the heat pipe fluid behavior.

The selected model uses a three dimensional transient conduction model with electrical heat generation in the bus bar and internal heat generation in the cells. This approach keeps the most important physics for the project: non uniform temperature, Joule heating, cell heating, conduction between parts, and convection to surrounding air. The embedded heat pipes are treated as effective high conductivity thermal features rather than full two phase devices.

Numerical method

Several numerical methods could be used to solve the governing equations. An explicit finite difference method would be simple to write, but the time step would need to be very small for stability in a transient three dimensional heat transfer model. A higher order Runge Kutta method could improve time accuracy, but it would require more calculations per time step. A finite element method could represent the geometry more easily, but it would go beyond the scope of this project. In general, numerical method selection requires balancing accuracy, stability, and computational cost (Chapra and Canale, 2015).

The selected method is a finite difference spatial model with a first order implicit time update. After the model is discretized in space, the transient heat equation becomes a large system of ordinary differential equations in time. The implicit method is used because it stays stable for the three dimensional thermal model while still being manageable to code. The linear system is solved using LU factorization so the same thermal matrix can be used efficiently over many time steps.

Overall, this method was selected because it gives a balance between physical realism, computational cost, and scope. It is detailed enough to compare the solid bus bar and modified bus bar designs, but still simple enough to explain and verify.

METHODS

This section includes the Math model and Numerical method.

Math model

The thermal behavior of the system is modeled as a coupled thermal electrical problem consisting of a copper bus bar and adjacent cylindrical battery cells. Temperature is treated as a function of position and time, $T(x, y, z, t)$, to capture non uniform heating and localized thermal effects.

Electrical current within the bus bar is governed by conservation of charge, which is a standard conservation-law basis for engineering mathematical models (Chapra and Canale, 2015),

$$\nabla \cdot (\sigma_e(T)\nabla V) = 0, \tag{1}$$

where V is the electric potential [V] and $\sigma_e(T)$ is the temperature dependent electrical conductivity [S/m]. The resulting current density is given by

$$\mathbf{J} = -\sigma_e(T)\nabla V, \tag{2}$$

and Joule heating within the conductor is

$$q'''_{\text{Joule}} = \sigma_e(T)\left|\nabla V\right|^2, \tag{3}$$

where q'''_{Joule} is the volumetric heat generation rate [W/m^3].

The transient temperature in the bus bar is governed by the heat conduction equation, which is commonly treated as a parabolic partial differential equation in numerical heat transfer problems (Chapra and Canale, 2015),

$$\rho_b c_{p,b}\frac{\partial T_b}{\partial t} = \nabla\cdot(k_b\nabla T_b) + q'''_{\text{Joule}}, \tag{4}$$

where ρ_b is density [kg/m^3], $c_{p,b}$ is specific heat [J/(kg $\cdot$ K)], and k_b is thermal conductivity [W/(m $\cdot$ K)].

Battery cells are modeled as cylindrical regions with internal heat generation due to electrical loading. The governing equation for the cells is

$$\rho_c c_{p,c}\frac{\partial T_c}{\partial t} = \nabla\cdot(k_c\nabla T_c) + q'''_{\text{cell}}, \tag{5}$$

where q'''_{cell} represents volumetric heat generation inside each cell [W/m^3].

Thermal coupling between the bus bar and cells occurs through conduction across their contact regions. Heat loss to the environment is modeled using convective boundary conditions,

$$-k\nabla T\cdot\mathbf{n} = h(T - T_\infty), \tag{6}$$

where h is the convection coefficient [$\mathrm{W}/(\mathrm{m}^2 \cdot \mathrm{K})$], T_∞ is ambient temperature [K], and $\mathbf{n}$ is the outward normal vector.

The effective ampacity of the system is defined as the maximum current that satisfies both temperature constraints,

$$T_{\max} \leq T_{\max,\mathrm{local}} \quad \text{and} \quad T_{\mathrm{avg}} \leq T_{\max,\mathrm{avg}}, \tag{7}$$

where $T_{\max,\mathrm{local}}$ is the allowable local maximum temperature and $T_{\max,\mathrm{avg}}$ is the allowable average temperature.

Numerical method

The governing equations form a coupled system of partial differential equations. Because the geometry, material regions, and heat generation terms make the system difficult to solve analytically, a numerical method is required.

The domain is discretized using a finite difference method on a structured three dimensional grid. Finite difference methods approximate derivatives at discrete points, allowing differential equations to be transformed into algebraic update equations (Chapra and Canale, 2015). Spatial derivatives are approximated using central differences, which are standard finite difference approximations for derivative terms in numerical differential equation models (Chapra and Canale, 2015). For example, the second derivative in the x direction is approximated as

$$\frac{\partial^2 T}{\partial x^2} \approx \frac{T_{i+1,j,k} - 2T_{i,j,k} + T_{i-1,j,k}}{\Delta x^2}, \tag{8}$$

where Δx is the grid spacing.

After spatial discretization, the thermal problem can be written as a large system of ordinary differential equations in time. This conversion is the reason the final transient model is solved as an ODE system rather than as a closed form analytical solution. Systems of ODEs are commonly solved by advancing all dependent variables through time using a numerical time integration method (Chapra and Canale, 2015),

$$\frac{d\mathbf{T}}{dt} = \mathbf{f}(\mathbf{T}, I, \mathbf{p}), \tag{9}$$

where $\mathbf{T}$ is the vector of nodal temperatures, I is current, and $\mathbf{p}$ represents model parameters such as thermal conductivity, density, heat capacity, and convection coefficient.

Time integration is performed using a first order implicit Euler update. Implicit Euler evaluates the derivative using the future unknown value, which improves stability compared with explicit Euler methods but requires solving a system of equations at each time step (Chapra and Canale, 2015). In simplified matrix form, the update is written as

$$(\mathbf{I} - \Delta t \mathbf{A})\, \mathbf{T}^{n+1} = \mathbf{T}^{n} + \Delta t \mathbf{b}, \tag{10}$$

where $\mathbf{I}$ is the identity matrix, Δt is the time step, $\mathbf{A}$ contains the discretized conduction and convection terms, $\mathbf{b}$ contains heat generation and ambient boundary terms, and $\mathbf{T}^{n+1}$ is the temperature vector at the next time step.

The electrical potential equation is solved as a sparse linear system to satisfy conservation of charge within the bus bar. Linear systems of equations commonly arise from discretized engineering models, and LU decomposition is useful when related linear systems must be solved efficiently (Chapra and Canale, 2015). The resulting electric field is used to compute Joule heating, which is then applied as a thermal source term during the transient heat solve.

The numerical method was selected because it provides stable time integration for the three dimensional heat transfer problem while keeping the code structure understandable and computationally manageable.

APPLICATION

This section includes the math model parameters, boundary conditions, numerical method parameters, and code outline. The values reported here define the specific implementation of the thermal electrical model for the 16 cell battery assembly and bus bar system.

The system consists of 16 cylindrical cells arranged in two rows of eight, with the second row horizontally offset to increase packing density. A copper bus bar is positioned adjacent to the cell pack and extends beyond one end. Two configurations are analyzed: a solid copper bus bar and a

modified bus bar containing embedded high conductivity regions representing integrated heat pipe features.

Math model parameters

The parameters used in the model are summarized in Table 1. Copper properties are used for the solid bus bar, while the embedded heat pipe regions are modeled as high effective conductivity regions. Battery cells are modeled using effective bulk thermal properties to capture pack level thermal behavior without resolving internal electrochemistry.

Table 1. Math model parameters for the 16 cell thermal electrical system. Geometry values are based on the modeled pack layout, while material and heat transfer values are taken from published copper, battery thermal management, bus bar, and heat pipe modeling sources.

Name	Symbol	Value	Units	Justification / Reference
Ambient temperature	T_∞	298.15	K	Room temperature.
Max local temperature	$T_{\max,local}$	353.15	K	80°C local limit.
Max average temperature	$T_{\max,avg}$	333.15	K	60°C average limit.
Cell diameter	D_c	0.0333	m	Modeled cell geometry.
Cell height	H_c	0.140	m	Modeled cell geometry.
Bus bar thickness	t_b	0.002	m	Selected conductor geometry.
Bus bar width	w_b	0.0621	m	Set by cell layout.
Heat pipe edge offset	s_{HP}	0.004	m	Sensitivity selected spacing.
Copper density	ρ_b	8960	$\mathrm{kg/m^3}$	Copper property (Cengel and Ghajar, 2015).
Copper specific heat	$c_{p,b}$	385	$\mathrm{J/(kg \cdot K)}$	Copper property (Cengel and Ghajar, 2015).
Copper thermal conductivity	k_b	400	$\mathrm{W/(m \cdot K)}$	Copper property (Incropera et al., 2011).
Cell density	ρ_c	2500	$\mathrm{kg/m^3}$	Effective cell value (Smith et al., 2018).
Cell specific heat	$c_{p,c}$	900	$\mathrm{J/(kg \cdot K)}$	Effective cell value (Smith et al., 2018).
Cell thermal conductivity	k_c	1.5	$\mathrm{W/(m \cdot K)}$	Effective cell value (Smith et al., 2018).
Heat pipe axial conductivity	$K_{HP,axial}$	15000	$\mathrm{W/(m \cdot K)}$	Effective heat pipe value (Yang et al., 2025; Wang et al., 2025).
Convection coefficient	h	15	$\mathrm{W/(m^2 \cdot K)}$	Natural convection estimate (Incropera et al., 2011).

Boundary conditions

The initial temperature of the system is uniform:

$$T(x, y, z, 0) = T_\infty. \tag{11}$$

Convective heat transfer is applied to exposed surfaces:

$$-k\nabla T \cdot \mathbf{n} = h(T - T_\infty). \tag{12}$$

Joule heating is applied within the electrically active bus bar:

$$q'''_{\text{Joule}} = \frac{J^2}{\sigma(T)}. \tag{13}$$

Internal heat generation is also applied within the cells as a current dependent volumetric source. Electrical boundary conditions are applied by assigning current input regions at the cell contact patches and a current output region at the bus bar output patch.

Numerical method parameters

The numerical parameters are summarized in Table 2. The governing PDE is discretized using finite differences, forming a system of ODEs in time that is solved using an implicit Euler update. This approach follows the standard numerical modeling process of replacing derivatives with finite difference approximations and then advances the resulting differential equation system through time (Chapra and Canale, 2015).

Table 2. Numerical method parameters used in the final model runs.

Name	Symbol	Value	Units	Justification
Grid spacing	$\Delta x, \Delta y, \Delta z$	0.003	m	Resolution vs cost
Time step	Δt	2.0	s	Convergence and runtime balance
Final time	t_f	600	s	Steady behavior check
Currents tested	I	300, 350, 400	A	Ampacity sweep
Local temperature limit	$T_{\text{max,local}}$	80	°C	Design constraint
Average temperature limit	$T_{\text{max,avg}}$	60	°C	Design constraint

A convergence study was performed on the heat pipe bus bar model at 350A to check whether the selected time step affected the main result. The final maximum temperature was selected as the convergence metric because it directly controls the local temperature limit used for ampacity evaluation. The baseline time step was $\Delta t = 2$ s, which is the value used for the final reported simulations. The tested time steps ranged from 1 s to 4 s, corresponding to −50% to +100% change relative to the baseline. The maximum change in final maximum temperature was only 0.0127%, so the selected time step was small enough that time discretization errors did not affect the ampacity conclusion.

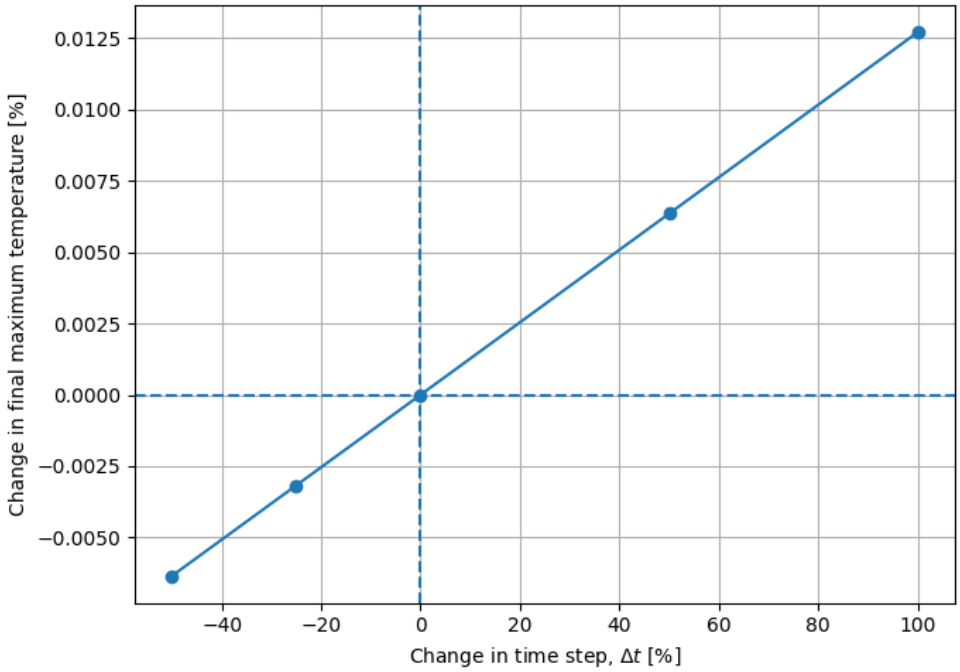


Figure 2. The actual heat pipe bus bar model shows negligible change in final maximum temperature when the time step is varied around the selected baseline value. Percent change in final maximum temperature is shown relative to the baseline time step of $\Delta t = 2.0$ s at 350 A.

Table 3. Actual model convergence results show that the final maximum temperature is insensitive to time step refinement near the selected baseline.

Δt [s]	Change in Δt [%]	T_{max} [°C]	T_{avg} [°C]	Change in T_{max} [%]
1.000	-50.000	58.555	57.305	-0.006342
1.500	-25.000	58.557	57.307	-0.003172
2.000	0.000	58.559	57.309	0.000000
3.000	50.000	58.563	57.313	0.006348
4.000	100.000	58.567	57.316	0.012702

Code outline

The numerical implementation follows these steps:

1. Construct the three dimensional geometry of the cell pack and bus bar.

2. Assign material properties to each region.

3. Build the electrical conductivity field for the selected configuration.

4. Solve the electrical potential equation for a 1 A reference case.
5. Scale the electrical solution to the desired current level.
6. Compute Joule heating in the bus bar.
7. Compute internal heat generation in the cells.
8. Assemble the implicit thermal matrix.
9. Advance the temperature field in time using the implicit update.
10. Record maximum and average temperatures.
11. Repeat for each current and bus bar configuration.
12. Perform sensitivity studies on selected uncertain or design dependent parameters.

RESULTS

The results are organized according to the objectives defined in the Introduction. Simulations were performed for both solid and heat pipe bus bar configurations at current levels of 300A, 350A, and 400A.

Objective 1: Temperature distribution

Objective 1 was addressed by computing the three dimensional temperature field for the modeled cell and bus bar assembly. The temperature distribution shows non uniform heating throughout the domain, with the highest temperatures occurring near the bus bar and cell contact regions. The solid bus bar and heat pipe bus bar temperature fields at 400A are shown in Figures 3 and 4.

Objective 2: Current vs. temperature response

Objective 2 was addressed by comparing maximum and average temperatures across the tested current levels. Temperature increased as current increased, which is consistent with Joule heating behavior. The heat pipe configuration reduced peak temperature compared to the solid bus bar at each tested current level, although the average temperature was slightly higher in the heat pipe

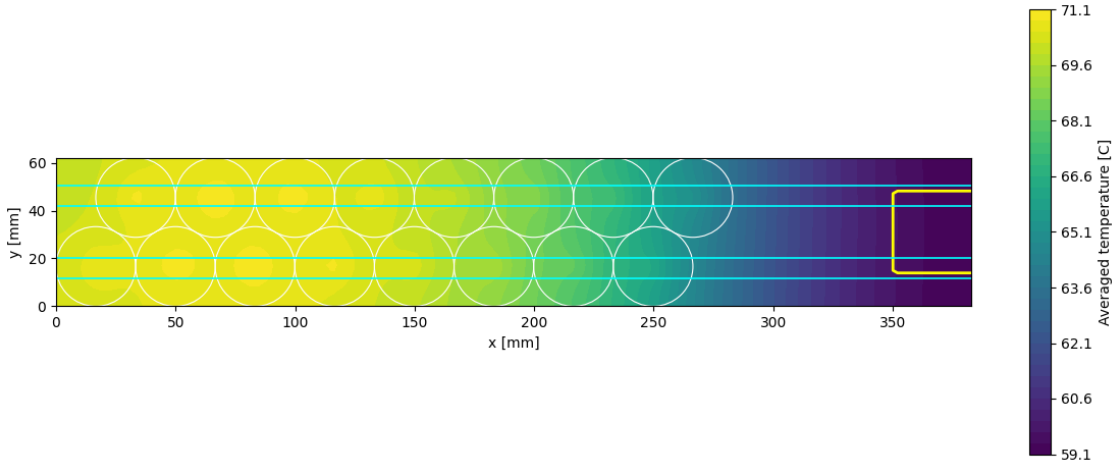


Figure 3. The solid bus bar reaches its highest temperatures near the contact and output regions at 400A.

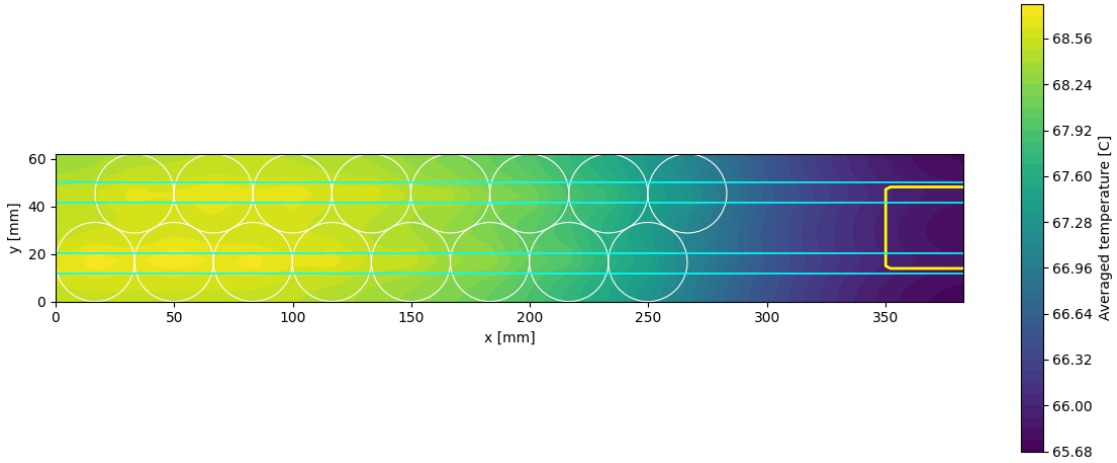


Figure 4. The heat pipe bus bar spreads heat more evenly along the bus bar and reduces the local peak temperature at 400A.

case because the heat pipe region spreads heat rather than removing additional total heat from the system.

The final current sweep results are summarized in Table 4. The temperature history at 400A is shown in Figure 6.

Objective 3: Effective ampacity

Objective 3 was addressed by comparing the maximum and average temperatures against the design limits. The local maximum temperature limit is 80°C, and the average temperature limit is

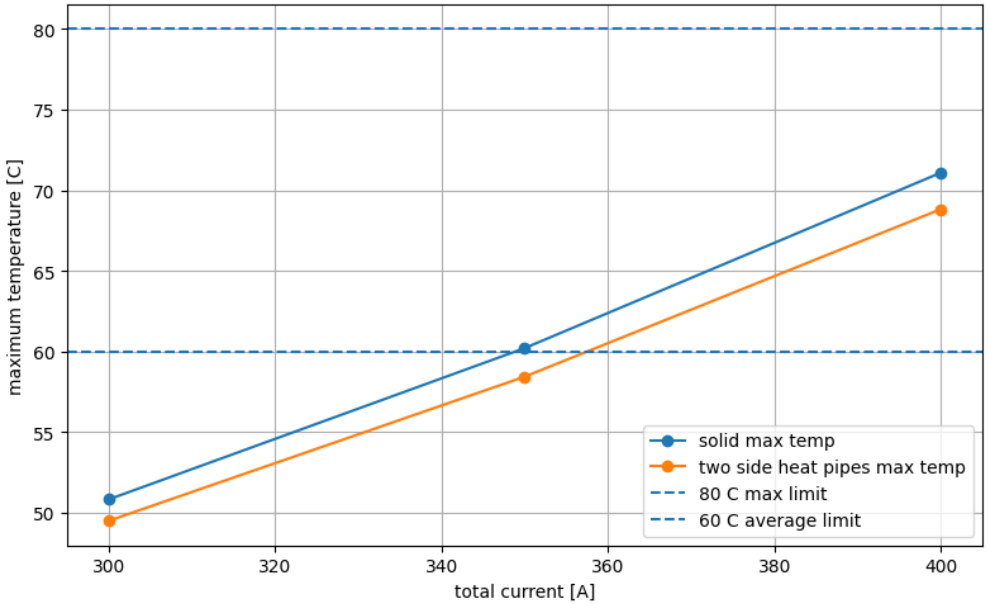


Figure 5. Maximum temperature increases with current for both bus bar configurations. The heat pipe configuration reduces peak temperature at each tested current level.

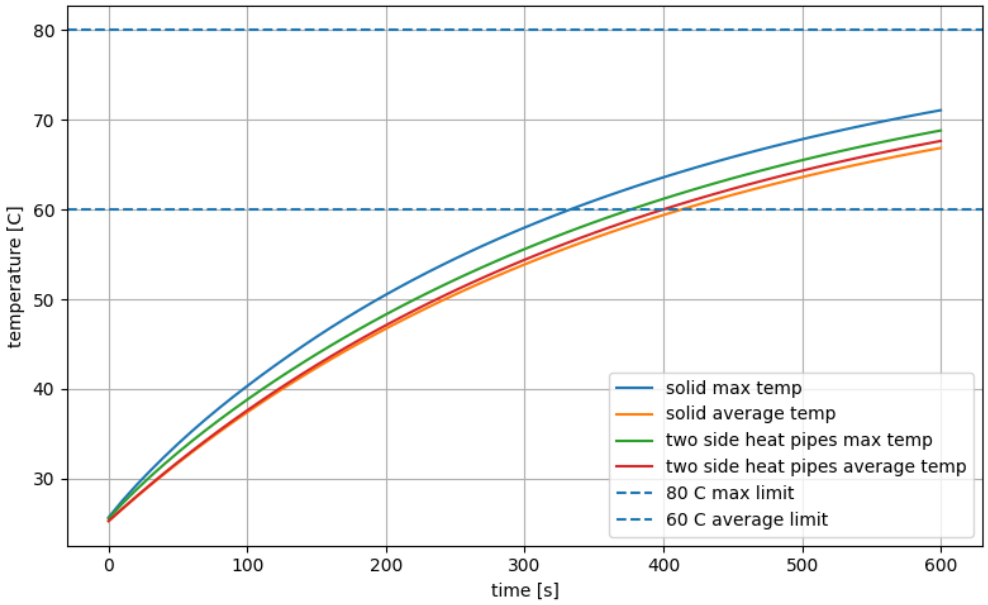


Figure 6. Temperature increases over time and approaches steady behavior during the 400A run.

60°C. Both the solid and heat pipe configurations pass both limits at 300 A and 350A. At 400A, both configurations remain below the 80°C local limit, but both exceed the 60°C average temperature limit.

Table 4. Final temperature results for solid and heat pipe bus bar configurations.

Current [A]	Solid T_{max} [°C]	Solid T_{avg} [°C]	HP T_{max} [°C]	HP T_{avg} [°C]	Result
300	50.808	48.388	49.480	48.805	Pass
350	60.199	56.936	58.426	57.519	Pass
400	71.081	66.868	68.819	67.652	Fails avg. limit

Therefore, under the tested conditions, the effective ampacity is at least 350A but below 400A for both configurations. The heat pipe design improves the peak temperature response, but it does not increase the tested ampacity above 400A because the average temperature limit is exceeded before the local maximum temperature limit.

Objective 4: Thermal interaction effects

Objective 4 was addressed by comparing the solid and heat pipe bus bar configurations. The heat pipe configuration reduces peak temperature by redistributing heat along the bus bar length. Current density plots are included because they help explain where the heat generation comes from (Figures 7 and 8). The heating plots show that the heat generation is concentrated around regions where current is crowded (Figures 9 and 10).

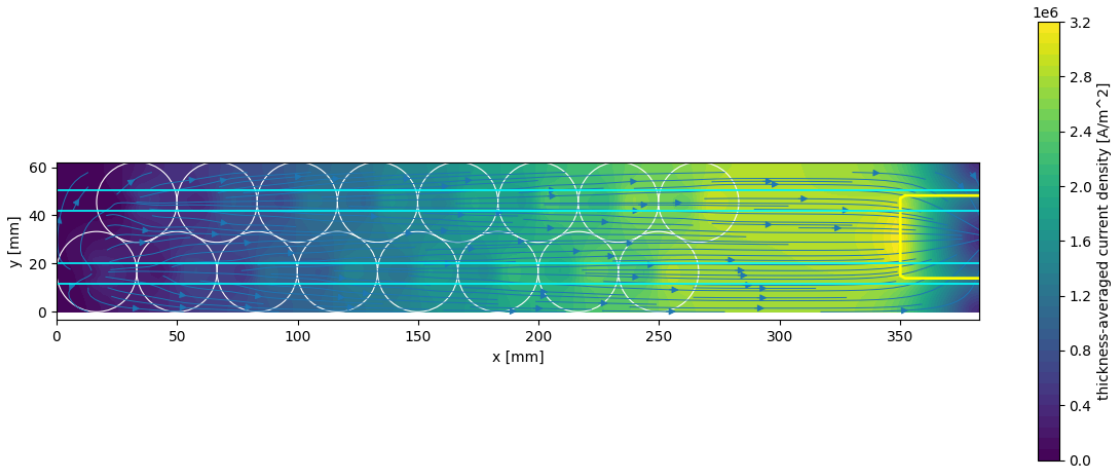


Figure 7. The solid bus bar current density field at 400A shows current crowding near the output region.

Because the embedded region does not remove heat from the system, the average temperature remains similar to the solid case and is slightly higher in the final current sweep. This distinction

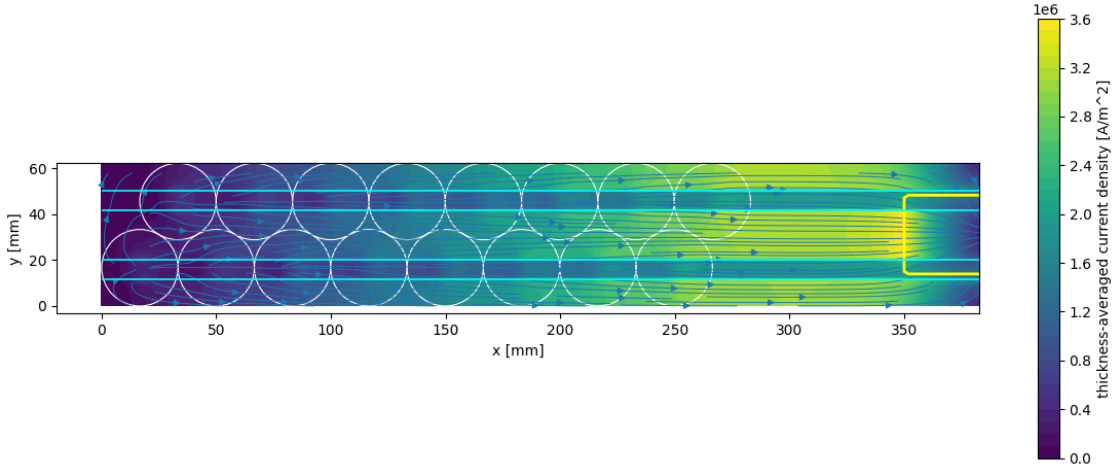


Figure 8. The heat pipe bus bar current density field at 400A shows how the modified material region changes the electrical loading pattern.

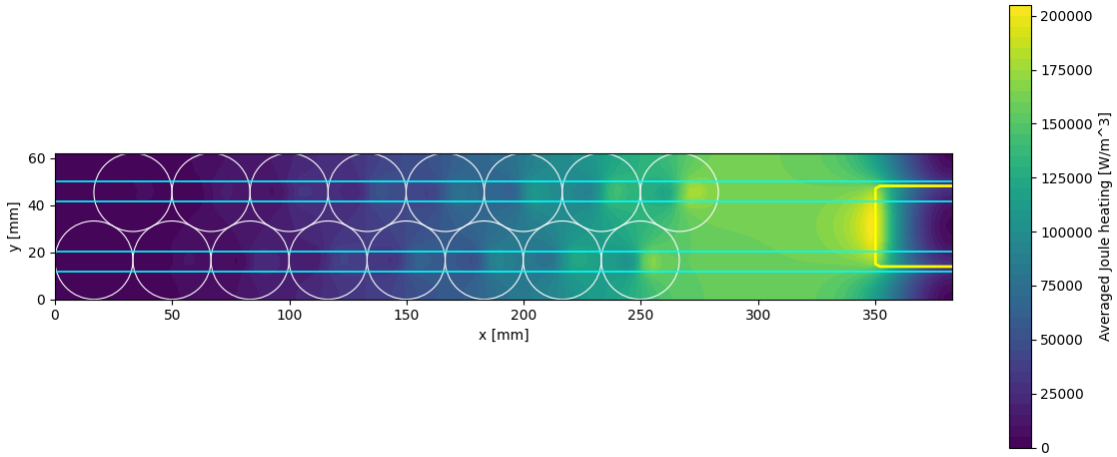


Figure 9. Joule heating in the solid bus bar is concentrated in regions with higher current density.

is important for design decisions. The heat pipe configuration is most useful for reducing local hot spots, but it does not replace the need for adequate system level heat rejection.

Summary of results

The primary findings are as follows:

- Peak temperatures occur near bus bar and cell contact regions.
- Temperature increases nonlinearly with current because Joule heating scales with current squared.

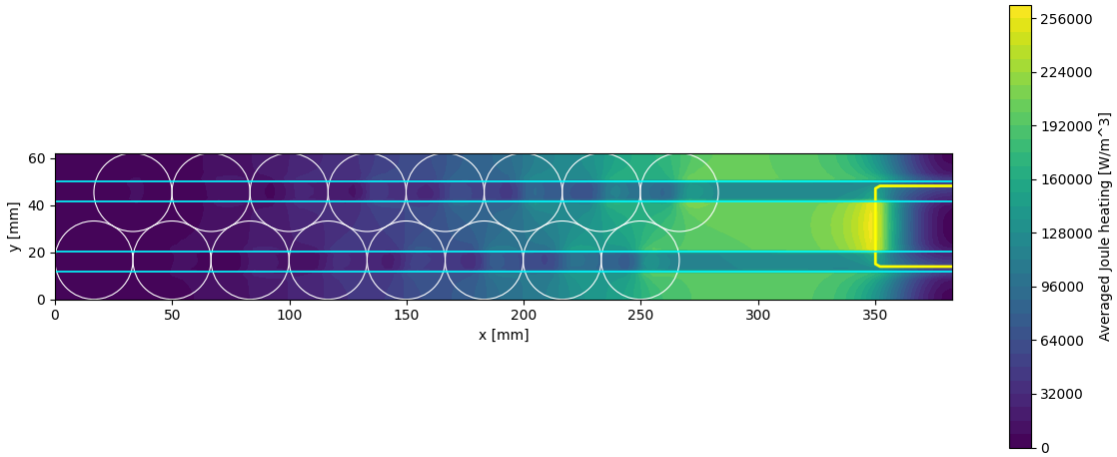


Figure 10. Joule heating in the heat pipe bus bar changes with the embedded high conductivity regions and current path.

- The heat pipe configuration reduces peak temperature compared to the solid bus bar at each tested current.
- Both configurations pass the 300A and 350A cases.
- Both configurations fail the 400A case because the average temperature exceeds the 60°C average temperature limit.
- The tested effective ampacity is at least 350A but below 400A.

DISCUSSION

This section interprets the results, discusses the sensitivity analysis, and identifies major sources of error.

Objective 1: Temperature distribution

The three dimensional temperature distributions obtained for both bus bar configurations match the expected behavior for a conductor with Joule heating and convective cooling. Peak temperatures occur near the contact patches, where current enters the bus bar through smaller areas. These regions create locally high current density, which increases Joule heating (Coneybeer et al., 1994). This makes sense because heat generation increases with the square of current density.

The results also show why a single average temperature would not be enough for this problem. A single average temperature would miss the hot spots near the contact regions. Since the local maximum temperature is one of the design limits, resolving the temperature field is necessary for evaluating effective ampacity.

Objective 2: Current vs. temperature response

The increase in temperature with current is consistent with the I^2R relationship that controls Joule heating. Increasing current from 300A to 400A raises the heat load substantially, causing both maximum and average temperature to increase. The model captures this expected behavior for both the solid and heat pipe configurations.

The heat pipe configuration reduces peak temperature at all tested currents. At 300A, the heat pipe case reduces peak temperature from 50.808°C to 49.480°C. At 400A, it reduces peak temperature from 71.081°C to 68.819°C. The improvement becomes larger at higher current, which suggests that axial heat spreading becomes more useful as heat generation increases.

Objective 3: Effective ampacity

Both configurations satisfy the 80°C local temperature limit at all tested currents, but both exceed the 60°C average temperature limit at 400A. Therefore, the average temperature limit controls the tested ampacity in this model. Based on the current sweep, the effective ampacity is at least 350A but below 400A.

The heat pipe configuration improves the peak temperature response but does not increase the tested ampacity above 400A because the failure condition is controlled by average temperature. This means the heat pipe is helpful for reducing hot spots, but overall heat removal still determines whether the design can meet the average temperature requirement at higher currents.

Objective 4: Thermal interaction effects

The temperature distribution in the cells is influenced by the bus bar through conductive coupling at the contact interface. The bus bar and cells should therefore be evaluated together instead of as

independent components. The results show that bus bar heating affects the temperature environment around the cells, and cell heat generation adds to the total thermal load.

The heat pipe configuration spreads heat more evenly along the bus bar. This reduces the maximum local temperature but does not significantly reduce average temperature. This is expected because the model represents the heat pipe as an enhanced conduction feature, not as active cooling. The heat pipe redistributes heat within the assembly, but it does not remove extra heat from the system.

Sensitivity analysis

The sensitivity analysis evaluated the influence of selected design and model parameters on peak temperature. The tested parameters were cell heat generation, heat pipe spacing from the bus bar edge, and heat pipe axial thermal conductivity. These parameters were selected because they represent either uncertain physical inputs or design features that could be changed in future bus bar layouts.

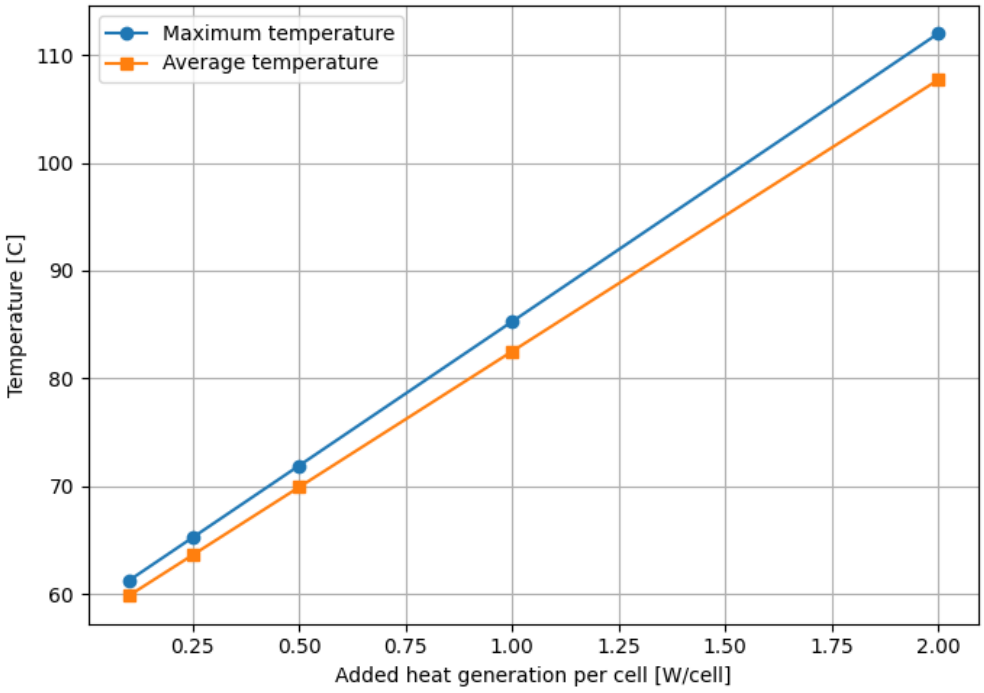


Figure 11. Increasing the cell heat generation rate increases both maximum and average temperature.

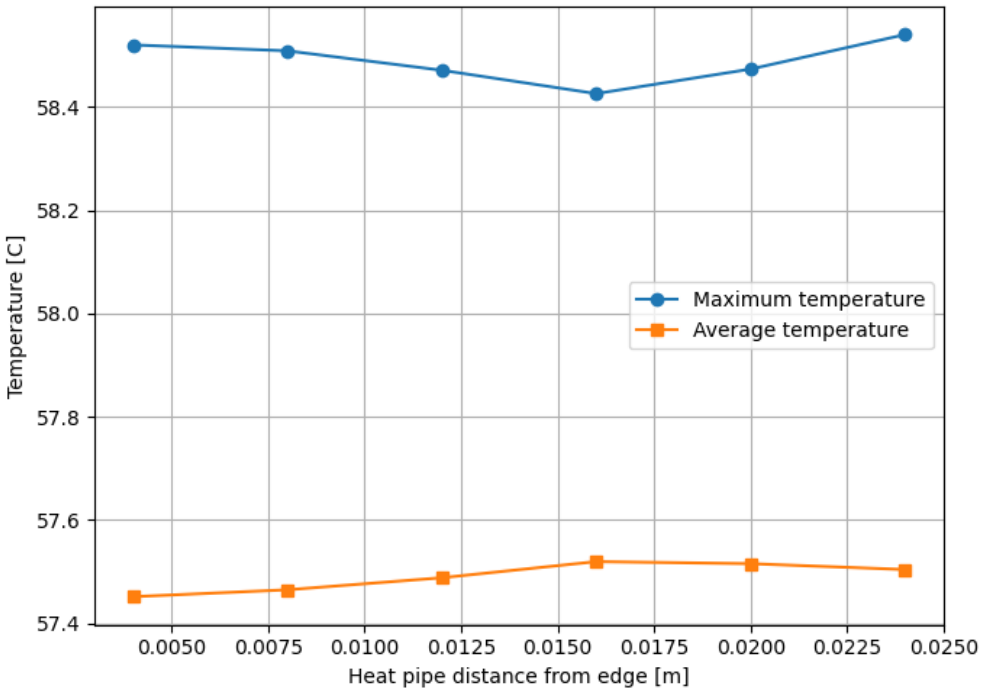


Figure 12. Heat pipe spacing affects peak temperature because placement changes how heat spreads through the bus bar.

The cell heat generation sensitivity showed that higher cell heat rates increase the overall temperature field. This is expected because cell heat generation adds thermal load to the system separate from bus bar Joule heating. However, cell heat generation should not be selected only to minimize temperature because it represents a physical operating condition rather than a design choice. For final design work, the cell heat rate should be based on measured or manufacturer data for the actual sodium ion cells.

The heat pipe spacing sensitivity showed that heat pipe placement affects peak temperature. Moving the high conductivity region changes how effectively heat is spread away from the contact and output regions. This makes heat pipe spacing a useful design variable, but it also has to be limited by manufacturability and by the need to leave enough copper around contact and weld regions.

The heat pipe axial conductivity sensitivity showed that increasing $K_{\mathrm{HP,axial}}$ reduces peak temperature. This confirms that the heat pipe benefit depends on achieving high effective axial

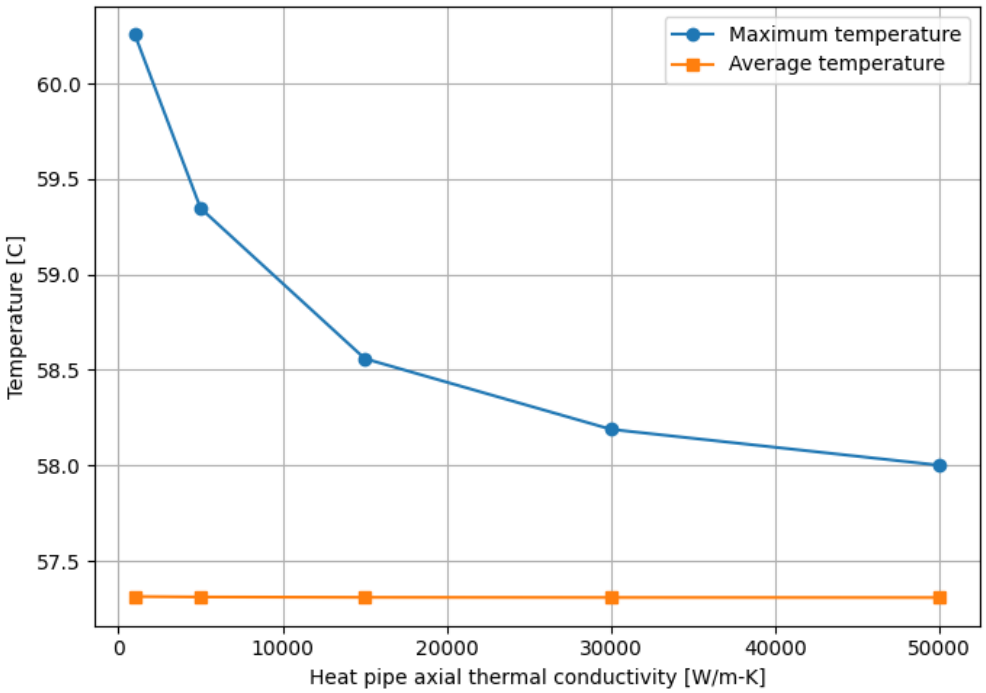


Figure 13. Higher heat pipe axial conductivity reduces peak temperature in the heat pipe bus bar case.

conductivity in the actual hardware. If the real heat pipe does not reach the assumed effective conductivity, the predicted peak temperature reduction would likely be smaller.

Overall, the sensitivity analysis supports the conclusion that the heat pipe configuration is most useful for reducing peak temperature, but it also shows that the final design should be tested physically before being used for production bus bar sizing.

Sources of error

Limitations of the mathematical model

The largest modeling simplification is that the heat pipes are represented as regions of elevated effective conductivity instead of full two phase fluid systems. Real heat pipes work through evaporation and condensation of a working fluid, and their performance depends on heat flux, orientation, wick design, and operating temperature (Yang et al., 2025). The effective conductivity used here is useful for comparing trends, but it is not a validated heat pipe model for this exact bus bar geometry.

The model also treats the battery cells using bulk effective properties rather than resolving internal electrochemistry, tab geometry, or internal winding structure. This is reasonable for a first system level model, but local cell temperatures near tabs or internal current collectors could be higher than the volume averaged cell temperature predicted by the model (Smith et al., 2018).

Parameter uncertainty

Several parameters were estimated because exact material and operating data were not available. Cell density, cell heat capacity, cell thermal conductivity, and cell heat generation are especially uncertain because they depend on the specific sodium ion cell construction and operating condition. The convection coefficient is also uncertain because actual motorcycle airflow and packaging could be very different from the natural convection assumption used in the model.

The sensitivity analysis shows that uncertain thermal parameters can influence the predicted peak temperature. Therefore, the model is more reliable for comparing design trends than for predicting exact absolute temperatures without validation.

Truncation error

The finite difference method is second order accurate in space for central difference conduction terms and first order accurate in time due to the implicit Euler update. The selected grid spacing of 3 mm and time step of 2 s were chosen to balance resolution and computational cost. The convergence study indicated that further refinement did not meaningfully change the primary conclusions.

Round off error

The implicit thermal solve uses matrix factorization to solve the linear system at each time step. No evidence of numerical instability or round off accumulation was observed in the reported simulations. Round off error is therefore not considered a major source of uncertainty compared with model simplifications and parameter uncertainty.

CONCLUSIONS

This project developed a three dimensional coupled thermal electrical model of a 16 cell bus bar assembly for an electric motorcycle application. The four project objectives were addressed as follows:

1. **Temperature distribution:** The model predicted a three dimensional temperature distribution for the 16 cell bus bar assembly. The results showed non uniform heating, with peak temperatures localized near cell contact regions. This supports the use of a spatially resolved model rather than a lumped average temperature model.

2. **Current vs. temperature response:** The model captured the expected relationship between current and temperature rise. Increasing current from 300A to 400A increased both maximum and average temperature, which is consistent with the I^2R dependence of Joule heating.

3. **Effective ampacity:** Both the solid and heat pipe configurations passed the 300A and 350A cases. Both configurations failed the 400A case because the average temperature exceeded the 60°C average temperature limit. Under the modeled conditions, the tested effective ampacity is at least 350A but below 400A.

4. **Thermal interaction effects:** The heat pipe configuration reduced peak temperature compared to the solid bus bar by redistributing heat axially. However, it did not significantly reduce average temperature because it spreads heat within the assembly rather than removing additional heat from the system.

The sensitivity analysis confirmed that heat pipe placement and heat pipe axial conductivity affect peak temperature. The results support continued development of the heat pipe bus bar concept, but the model should be validated with physical testing before it is used for final design decisions.

RECOMMENDATIONS

The model results are useful for comparing early bus bar design options, but they are not enough for final production sizing without physical validation. Based on the results, sensitivity analysis, and sources of error, the following actions are recommended:

- Build both a solid bus bar prototype and an embedded heat pipe bus bar design using the modeled geometry. Testing both designs under the same current profile would show whether the predicted peak temperature reduction occurs in real hardware.
- Test the prototypes at 300A, 350A, and 400A while measuring temperature with thermocouples and a thermal camera. The model predicts that both designs pass at 350A but fail the average temperature limit at 400A, so these current levels are the most useful for validation.
- Measure temperatures near cell contact regions, bus bar centerline locations, and the output patch. These are the locations most likely to show local hot spots, so they are the most important locations for checking the model.
- Measure or estimate cell heat generation using the actual sodium ion cells intended for the motorcycle. Cell heat generation is a physical input, not a design variable, so the final model should use measured cell data instead of a broad estimate.
- Use the heat pipe bus bar concept mainly to reduce local hot spots, not as the only method for controlling average temperature. Since the 400A case fails because of average temperature, future design work should also consider improved convection, increased cooling surface area, or forced airflow.
- Treat the current model as a design comparison tool until validation data are available. The model is useful for deciding which bus bar configurations are worth prototyping, but final ampacity ratings should be based on measured temperatures from physical testing.

References

Cengel, Y. A. and Ghajar, A. J. (2015). *Heat and Mass Transfer: Fundamentals and Applications*. McGraw-Hill Education, New York, 5 edition.

Chapra, S. C. and Canale, R. P. (2015). *Numerical Methods for Engineers*. McGraw-Hill Education, New York, NY, 7 edition.

Coneybeer, R., Black, W., and Bush, R. (1994). "Steady-state and transient ampacity of bus bar." *IEEE Transactions on Power Delivery*, 9(4), 1822–1829.

Incropera, F. P., DeWitt, D. P., Bergman, T. L., and Lavine, A. S. (2011). *Fundamentals of Heat and Mass Transfer*. John Wiley and Sons, Hoboken, New Jersey, 7 edition.

Smith, J., Singh, R., Hinterberger, M., and Mochizuki, M. (2018). "Battery thermal management system for electric vehicle using heat pipes." *International Journal of Thermal Sciences*, 134, 517–529.

Wang, R., Zhao, J., Chen, J., Ding, R., and Luo, D. (2025). "Integrated design and optimization of a heat pipe-based thermoelectric generator for range-extended electric vehicles." *Thermal Science and Engineering Progress*, 67, 104159.

Yang, L., Zhang, Z., Xian, Y., Bai, X., Bian, X., Li, Y., and Tao, H. (2025). "Enhancing thermal diffusion in busbars through heat pipe coupling: A novel approach for alleviating contact overheating in power distribution systems." *Applied Thermal Engineering*, 258, 124683.

APPENDIX I. PROGRAM VERIFICATION

Program verification was performed to check whether the numerical time stepping method was implemented correctly. Verification is different from validation. Verification checks whether the code solves a known mathematical problem correctly, while validation checks whether the model represents the real physical system accurately.

The verification problem used Newton's law of cooling,

$$\frac{dT}{dt} = -a(T - T_\infty), \tag{14}$$

where T is temperature [°C], T_∞ is ambient temperature [°C], and a is a cooling coefficient [s^{-1}]. This problem was selected because it has the same transient decay behavior as a thermal model but has a known analytical solution.

Using a problem with a known analytical solution provides a direct check on whether the numerical update is implemented correctly, since the numerical result can be compared against the exact solution as the step size is refined (Chapra and Canale, 2015).

The analytical solution is

$$T(t) = T_\infty + (T_0 - T_\infty)e^{-at}, \tag{15}$$

where T_0 is the initial temperature. The numerical solution was computed using the same first order implicit Euler approach used in the main thermal model. For this verification problem, the implicit update is

$$T^{n+1} = \frac{T^n + \Delta t a T_\infty}{1 + \Delta t a}. \tag{16}$$

The numerical and analytical solutions were compared for several time step sizes. As the time step decreased, the numerical solution approached the analytical solution (Figure I.1). The final temperature percent error also decreased with time step refinement (Figure I.2), confirming that the time stepping method was implemented correctly.

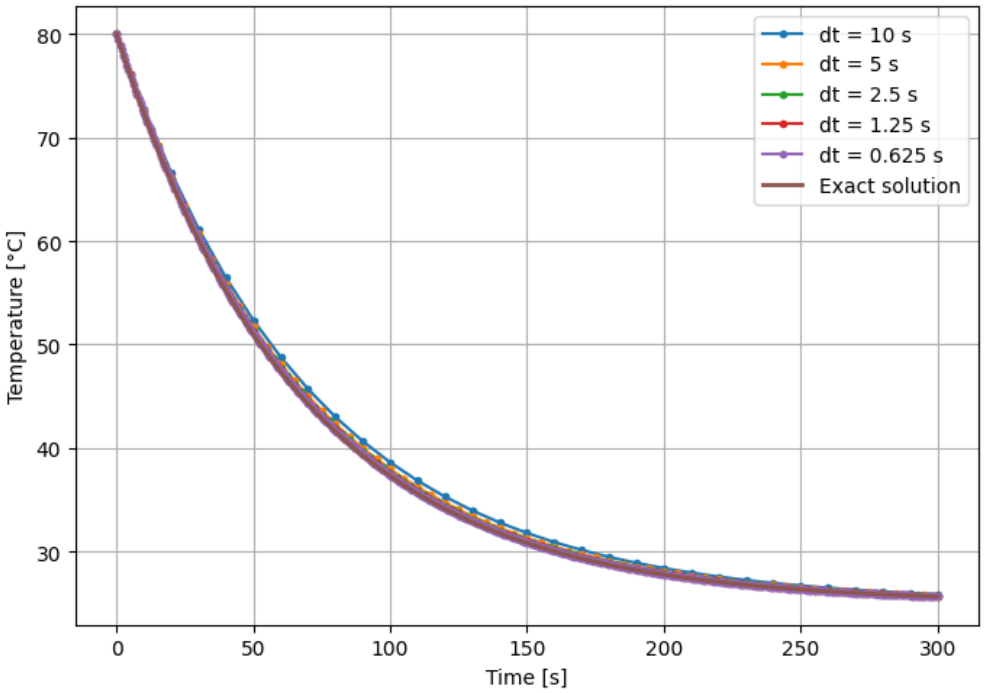


Figure I.1. Numerical temperature predictions approach the analytical Newton cooling solution as the time step decreases, verifying the transient thermal update.

Table I.1. Verification error decreases as the time step is refined.

Time step Δt [s]	Final temperature error [%]
10.000	0.8577
5.000	0.4161
2.500	0.2047
1.250	0.1015
0.625	0.0505

The decreasing error confirms that the numerical update behaves as expected for a known analytical solution. This verifies the time stepping portion of the program, but it does not validate the physical assumptions of the full bus bar model.

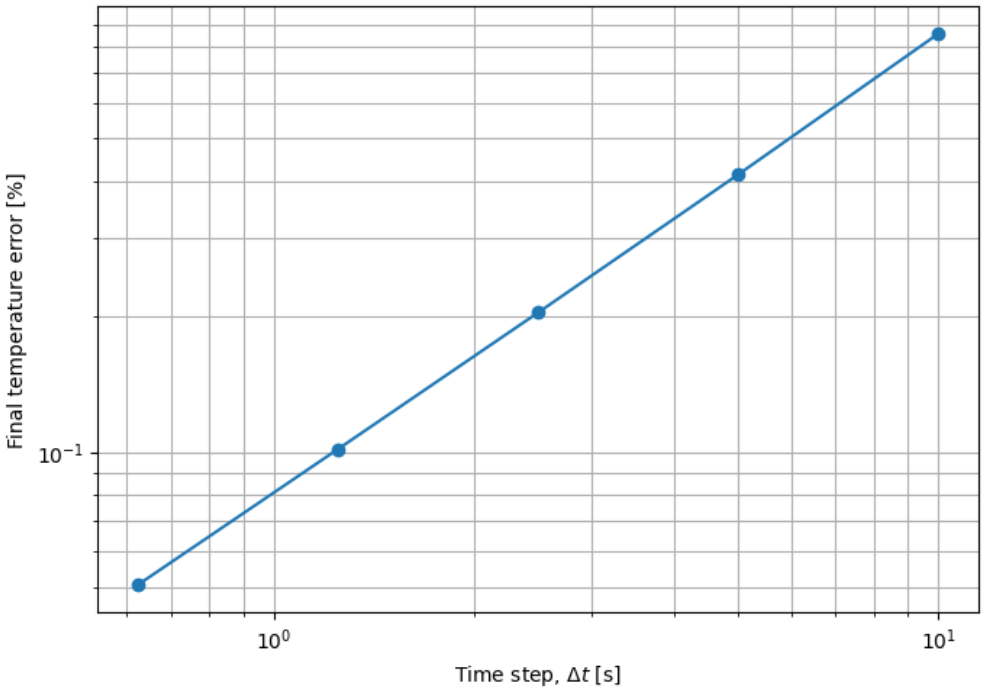


Figure I.2. Final temperature error decreases as the time step is refined, showing convergence of the implicit Euler verification problem.

APPENDIX II. PROGRAM SOURCE CODE

The full Python source code used for the model is included below. The code constructs the geometry, solves the electrical field, computes Joule and cell heat generation, solves the transient thermal model, performs sensitivity analysis, and generates the final figures used in this report.

```
import numpy as np
import matplotlib.pyplot as plt
import time as time_module


from scipy.sparse import lil_matrix
from scipy.sparse import csc_matrix
from scipy.sparse import csr_matrix


from scipy.sparse.linalg import spsolve
from scipy.sparse.linalg import splu
```

```
"""***Inputs***"""

START_CURRENT_A = 300.0
CURRENT_STEP_A = 50.0
MAX_CURRENT_A = 400.0

T_MAX_LOCAL_K = 353.15
T_MAX_AVG_K = 333.15

CELL_D = 33.3e-3        # [m]
CELL_R = CELL_D / 2.0   # [m]

N_CELLS_X = 8
N_CELLS_Y = 2
N_CELLS_TOTAL = N_CELLS_X * N_CELLS_Y

ROW_Y_PITCH = np.sqrt(3.0) / 2.0 * CELL_D
ROW_X_OFFSET = 0.5 * CELL_D

BUSBAR_W = CELL_D + ROW_Y_PITCH
CELL_REGION_L = (N_CELLS_X + 0.5) * CELL_D

BUSBAR_T = 2.0e-3       # [m]
BUSBAR_OVERHANG = 3.0 * CELL_D
BUSBAR_L = CELL_REGION_L + BUSBAR_OVERHANG

# Cell Contacts
CONTACT_DIAMETER = 10.0e-3 # [m]
```

```
CONTACT_R = CONTACT_DIAMETER / 2.0

# Current Output
OUTPUT_LENGTH = 1.0 * CELL_D
OUTPUT_WIDTH = 1.0 * CELL_D

# Mesh Size
NX = 100
NY = 30
NZ = 10

"""***Heat Pipes***"""

USE_HEAT_PIPE = True

N_HEAT_PIPES = 2

HP_WIDTH = 8.0e-3       # [m]
HP_HEIGHT = 1.0e-3      # [m]

# For Both
HP_EDGE_OFFSET = 2.0e-3 # [m]

# Effective heat pipe conductivities
K_HP_AXIAL = 15000.0    # [W/m-K], along busbar length
K_HP_TRANS = 100.0      # [W/m-K], along width

# Electrical conduction heat pipe region [1=Copper]
HP_ELECTRICAL_RELATIVE = 0.02
```

```
"""***Physical Constriants***"""

# Ambient / convection
T_INF = 298.15          # [K] 25 C

H_TOP = 15.0            # [W/m^2-K]
H_BOTTOM = 5.0          # [W/m^2-K]
H_SIDE = 10.0           # [W/m^2-K]

# Copper properties
SIGMA20_CU = 58.0e6     # [S/m]
ALPHA20_CU = 0.00393    # [1/K]
T_REF = 293.15          # [K]

RHO_CU = 8960.0         # [kg/m^3]
CP_CU = 385.0           # [J/kg-K]
K_CU = 401.0            # [W/m-K]


# Cell Contact heating
CELL_INTERNAL_R = 1.5e-3 # [ohm/cell]
WELD_R = 0.8e-3         # [ohm/contact]
CELL_FIXED_HEAT_W = 0.0 # [W/cell]

"""***Time***"""

DT = 2.0
T_FINAL = 600.0
```

```
STEADY_TOL_K = 2.0e-5
MIN_STEPS_BEFORE_STEADY_CHECK = 20

SAVE_EVERY = 5
PRINT_EVERY = 40

"""***Basics***"""

def sigma_cu(T_K):
    return SIGMA20_CU / (1.0 + ALPHA20_CU * (T_K - T_REF))



def harmonic_mean(a, b, eps=1e-30):
    return 2.0 * a * b / (a + b + eps)



def node_id(i, j, k):
    return (i * NY + j) * NZ + k

"""***Gradient***"""

def gradient_3d(F, dx, dy, dz):
    dFdx = np.zeros_like(F)
    dFdy = np.zeros_like(F)
    dFdz = np.zeros_like(F)

    dFdx[1:-1, :, :] = (F[2:, :, :] - F[:-2, :, :]) / (2.0 * dx)
    dFdx[0, :, :] = (F[1, :, :] - F[0, :, :]) / dx
    dFdx[-1, :, :] = (F[-1, :, :] - F[-2, :, :]) / dx
```

```
    dFdy[:, 1:-1, :] = (F[:, 2:, :] - F[:, :-2, :]) / (2.0 * dy)
    dFdy[:, 0, :] = (F[:, 1, :] - F[:, 0, :]) / dy
    dFdy[:, -1, :] = (F[:, -1, :] - F[:, -2, :]) / dy

    dFdz[:, :, 1:-1] = (F[:, :, 2:] - F[:, :, :-2]) / (2.0 * dz)
    dFdz[:, :, 0] = (F[:, :, 1] - F[:, :, 0]) / dz
    dFdz[:, :, -1] = (F[:, :, -1] - F[:, :, -2]) / dz

    return dFdx, dFdy, dFdz

"""***Geometry***"""

def build_geometry():
    x = np.linspace(0.0, BUSBAR_L, NX)
    y = np.linspace(0.0, BUSBAR_W, NY)
    z = np.linspace(0.0, BUSBAR_T, NZ)

    dx = x[1] - x[0]
    dy = y[1] - y[0]
    dz = z[1] - z[0]

    X, Y, Z = np.meshgrid(x, y, z, indexing="ij")

    busbar_mask = np.ones((NX, NY, NZ), dtype=bool)

    bottom_layer = np.zeros_like(busbar_mask)
    bottom_layer[:, :, 0] = True
```

```
top_layer = np.zeros_like(busbar_mask)
top_layer[:, :, -1] = True

centers = []

y_bottom = CELL_R
y_top = CELL_R + ROW_Y_PITCH

for i in range(N_CELLS_X):
    cx_bottom = (i + 0.5) * CELL_D
    cy_bottom = y_bottom
    centers.append((cx_bottom, cy_bottom, "bottom"))

    cx_top = (i + 0.5) * CELL_D + ROW_X_OFFSET
    cy_top = y_top
    centers.append((cx_top, cy_top, "top"))

cell_source_masks = []

for cx, cy, row_name in centers:
    r2 = (X - cx) ** 2 + (Y - cy) ** 2
    pad = (r2 <= CONTACT_R ** 2) & bottom_layer
    cell_source_masks.append(pad)

any_source = np.zeros_like(busbar_mask)

for pad in cell_source_masks:
    any_source |= pad
```

```
# Output

output_x_min = BUSBAR_L - OUTPUT_LENGTH
output_y_center = BUSBAR_W / 2.0
output_y_min = output_y_center - OUTPUT_WIDTH / 2.0
output_y_max = output_y_center + OUTPUT_WIDTH / 2.0

output_mask = (
    (X >= output_x_min) &
    (Y >= output_y_min) &
    (Y <= output_y_max) &
    bottom_layer)

hp_mask = np.zeros_like(busbar_mask, dtype=bool)

if USE_HEAT_PIPE:
    if N_HEAT_PIPES == 1:
        hp_y_centers = [
            BUSBAR_W / 2.0]

    elif N_HEAT_PIPES == 2:
        hp_y_centers = [
            HP_EDGE_OFFSET,
            BUSBAR_W - HP_EDGE_OFFSET]

    else:
        raise ValueError(
            "N_HEAT_PIPES must be 1 or 2. "
            "Use N_HEAT_PIPES = 1 for one center heat pipe, "
```

```
            "or N_HEAT_PIPES = 2 for two side heat pipes.")

    hp_z_center = BUSBAR_T / 2.0

    for hp_y_center in hp_y_centers:
        this_hp = (
            (Y >= hp_y_center - HP_WIDTH / 2.0) &
            (Y <= hp_y_center + HP_WIDTH / 2.0) &
            (Z >= hp_z_center - HP_HEIGHT / 2.0) &
            (Z <= hp_z_center + HP_HEIGHT / 2.0))

        hp_mask |= this_hp

return {
    "x": x,
    "y": y,
    "z": z,
    "dx": dx,
    "dy": dy,
    "dz": dz,
    "X": X,
    "Y": Y,
    "Z": Z,
    "busbar_mask": busbar_mask,
    "bottom_layer": bottom_layer,
    "top_layer": top_layer,
    "centers": centers,
    "cell_source_masks": cell_source_masks,
    "any_source": any_source,
```

```
        "output_mask": output_mask,
        "hp_mask": hp_mask,}

"""***Electric / Thermal Feilds***"""

def build_relative_electrical_field(geom, case_name):
    rel = np.ones((NX, NY, NZ), dtype=float)

    if "heat" in case_name.lower() and USE_HEAT_PIPE:
        rel[geom["hp_mask"]] = HP_ELECTRICAL_RELATIVE

    rel = np.maximum(rel, 1.0e-8)

    return rel


def build_thermal_fields(geom, case_name):
    kx = np.full((NX, NY, NZ), K_CU, dtype=float)
    ky = np.full((NX, NY, NZ), K_CU, dtype=float)
    kz = np.full((NX, NY, NZ), K_CU, dtype=float)

    if "heat" in case_name.lower() and USE_HEAT_PIPE:
        hp = geom["hp_mask"]
        kx[hp] = K_HP_AXIAL
        ky[hp] = K_HP_TRANS
        kz[hp] = K_HP_TRANS

    return kx, ky, kz
```

```
"""***Electric 1***"""

def solve_base_electrical_1A(geom, case_name):
    dx = geom["dx"]
    dy = geom["dy"]
    dz = geom["dz"]

    rel_sigma = build_relative_electrical_field(geom, case_name)

    N = NX * NY * NZ
    A = lil_matrix((N, N), dtype=float)
    b = np.zeros(N, dtype=float)

    I_cell_base = 1.0 / N_CELLS_TOTAL

    # Positive current enters
    for pad in geom["cell_source_masks"]:
        ids = np.flatnonzero(pad.ravel())
        b[ids] += I_cell_base / len(ids)

    # Negative current exits
    out_ids = np.flatnonzero(geom["output_mask"].ravel())
    b[out_ids] -= 1.0 / len(out_ids)

    print(f"Building base electrical matrix for {case_name}...")
    t0 = time_module.time()

    for i in range(NX):
        for j in range(NY):
```

```
for k in range(NZ):
    p = node_id(i, j, k)
    diag = 0.0

    if i > 0:
        gf = harmonic_mean(rel_sigma[i, j, k], rel_sigma[i - 1, j, k])
        G = gf * dy * dz / dx
        A[p, node_id(i - 1, j, k)] = -G
        diag += G

    if i < NX - 1:
        gf = harmonic_mean(rel_sigma[i, j, k], rel_sigma[i + 1, j, k])
        G = gf * dy * dz / dx
        A[p, node_id(i + 1, j, k)] = -G
        diag += G

    if j > 0:
        gf = harmonic_mean(rel_sigma[i, j, k], rel_sigma[i, j - 1, k])
        G = gf * dx * dz / dy
        A[p, node_id(i, j - 1, k)] = -G
        diag += G

    if j < NY - 1:
        gf = harmonic_mean(rel_sigma[i, j, k], rel_sigma[i, j + 1, k])
        G = gf * dx * dz / dy
        A[p, node_id(i, j + 1, k)] = -G
        diag += G

    if k > 0:
```

```
                gf = harmonic_mean(rel_sigma[i, j, k], rel_sigma[i, j, k - 1])
                G = gf * dx * dy / dz
                A[p, node_id(i, j, k - 1)] = -G
                diag += G

            if k < NZ - 1:
                gf = harmonic_mean(rel_sigma[i, j, k], rel_sigma[i, j, k + 1])
                G = gf * dx * dy / dz
                A[p, node_id(i, j, k + 1)] = -G
                diag += G

            A[p, p] = diag

# Pin one node
pin = node_id(NX - 1, NY - 1, NZ - 1)
A[pin, :] = 0.0
A[pin, pin] = 1.0
b[pin] = 0.0

print(f"Electrical matrix built in {time_module.time() - t0:.2f} s")
print(f"Solving base electrical field for {case_name}...")

t1 = time_module.time()
phi = spsolve(csr_matrix(A), b).reshape((NX, NY, NZ))

print(f"Base electrical solve finished in {time_module.time() - t1:.2f} s")

dphidx, dphidy, dphidz = gradient_3d(phi, dx, dy, dz)
grad_phi_2 = dphidx ** 2 + dphidy ** 2 + dphidz ** 2
```

```
    return {
        "case_name": case_name,
        "rel_sigma": rel_sigma,
        "phi": phi,
        "dphidx": dphidx,
        "dphidy": dphidy,
        "dphidz": dphidz,
        "grad_phi_2": grad_phi_2,}

"""***Electric 2***"""

def electrical_fields_from_base(ebase, current_A, T_mean_K):
    sigma = sigma_cu(T_mean_K)
    rel_sigma = ebase["rel_sigma"]

    V = ebase["phi"] * current_A / sigma

    Jx = -rel_sigma * current_A * ebase["dphidx"]
    Jy = -rel_sigma * current_A * ebase["dphidy"]
    Jz = -rel_sigma * current_A * ebase["dphidz"]

    q_joule = (current_A ** 2 / sigma) * rel_sigma * ebase["grad_phi_2"]

    return V, Jx, Jy, Jz, q_joule, sigma

"""***Heating***"""

def add_convection_to_node(i, j, k, dx, dy, dz):
```

```
    H_total = 0.0

    if i == 0:
        H_total += H_SIDE * dy * dz

    if i == NX - 1:
        H_total += H_SIDE * dy * dz

    if j == 0:
        H_total += H_SIDE * dx * dz

    if j == NY - 1:
        H_total += H_SIDE * dx * dz

    if k == 0:
        H_total += H_BOTTOM * dx * dy

    if k == NZ - 1:
        H_total += H_TOP * dx * dy

    return H_total



def build_thermal_lu(geom, case_name):
    dx = geom["dx"]
    dy = geom["dy"]
    dz = geom["dz"]

    dvol = dx * dy * dz
```

```
C = RHO_CU * CP_CU * dvol

kx, ky, kz = build_thermal_fields(geom, case_name)

N = NX * NY * NZ
A = lil_matrix((N, N), dtype=float)

print(f"Building thermal matrix for {case_name}..")
t0 = time_module.time()

for i in range(NX):
    for j in range(NY):
        for k in range(NZ):
            p = node_id(i, j, k)
            diag = C / DT

            if i > 0:
                k_face = harmonic_mean(kx[i, j, k], kx[i - 1, j, k])
                G = k_face * dy * dz / dx
                A[p, node_id(i - 1, j, k)] = -G
                diag += G

            if i < NX - 1:
                k_face = harmonic_mean(kx[i, j, k], kx[i + 1, j, k])
                G = k_face * dy * dz / dx
                A[p, node_id(i + 1, j, k)] = -G
                diag += G

            if j > 0:
```

```
            k_face = harmonic_mean(ky[i, j, k], ky[i, j - 1, k])
            G = k_face * dx * dz / dy
            A[p, node_id(i, j - 1, k)] = -G
            diag += G

        if j < NY - 1:
            k_face = harmonic_mean(ky[i, j, k], ky[i, j + 1, k])
            G = k_face * dx * dz / dy
            A[p, node_id(i, j + 1, k)] = -G
            diag += G

        if k > 0:
            k_face = harmonic_mean(kz[i, j, k], kz[i, j, k - 1])
            G = k_face * dx * dy / dz
            A[p, node_id(i, j, k - 1)] = -G
            diag += G

        if k < NZ - 1:
            k_face = harmonic_mean(kz[i, j, k], kz[i, j, k + 1])
            G = k_face * dx * dy / dz
            A[p, node_id(i, j, k + 1)] = -G
            diag += G

        H = add_convection_to_node(i, j, k, dx, dy, dz)
        diag += H

        A[p, p] = diag

print(f"Thermal matrix built in {time_module.time() - t0:.2f} s")
```

```
    print(f"Factorizing thermal matrix for {case_name}...")

    t1 = time_module.time()
    lu = splu(csc_matrix(A))

    print(f"Thermal factorization finished in {time_module.time() - t1:.2f} s")

    return lu

def thermal_rhs(geom, T, q_total):
    dx = geom["dx"]
    dy = geom["dy"]
    dz = geom["dz"]

    dvol = dx * dy * dz
    C = RHO_CU * CP_CU * dvol

    rhs = (C / DT) * T.ravel() + q_total.ravel() * dvol

    for i in range(NX):
        for j in range(NY):
            for k in range(NZ):
                H = add_convection_to_node(i, j, k, dx, dy, dz)

                if H > 0.0:
                    p = node_id(i, j, k)
                    rhs[p] += H * T_INF

    return rhs
```

```
def make_cell_heat_source(geom, current_A):
    q_cell = np.zeros((NX, NY, NZ), dtype=float)

    I_cell = current_A / N_CELLS_TOTAL

    P_internal = I_cell ** 2 * CELL_INTERNAL_R
    P_weld = I_cell ** 2 * WELD_R
    P_fixed = CELL_FIXED_HEAT_W

    P_cell_total = P_internal + P_weld + P_fixed

    dvol = geom["dx"] * geom["dy"] * geom["dz"]

    for pad in geom["cell_source_masks"]:
        n_nodes = np.count_nonzero(pad)

        if n_nodes > 0:
            q_cell[pad] += P_cell_total / (n_nodes * dvol)

    return q_cell

"""***Thermal Matrix***

***Run***
"""

def run_case(geom, ebase, thermal_lu, current_A, case_name):
    print("\n" + "=" * 80)
```

```
print(f"Running {case_name} at {current_A:.1f} A")
print("=" * 80)

T = np.full((NX, NY, NZ), T_INF, dtype=float)
q_cell = make_cell_heat_source(geom, current_A)

n_steps = int(T_FINAL / DT)

times = []
Tmax_hist = []
Tavg_hist = []

V = None
Jx = None
Jy = None
Jz = None
q_joule = None
sigma_now = None

last_print = time_module.time()

for n in range(n_steps + 1):
    T_old = T.copy()
    T_mean = np.mean(T)

    V, Jx, Jy, Jz, q_joule, sigma_now = electrical_fields_from_base(
        ebase,
        current_A,
        T_mean)
```

```
q_total = q_joule + q_cell


rhs = thermal_rhs(geom, T, q_total)
T = thermal_lu.solve(rhs).reshape((NX, NY, NZ))


max_change = np.max(np.abs(T - T_old))


if n % SAVE_EVERY == 0 or n == n_steps:
    times.append(n * DT)
    Tmax_hist.append(np.max(T))
    Tavg_hist.append(np.mean(T))


if n % PRINT_EVERY == 0 or n == n_steps:
    now = time_module.time()


    print(
        f"{case_name:>18s} | "
        f"step {n:5d}/{n_steps} | "
        f"t = {n * DT:8.1f} s | "
        f"Tmax = {np.max(T) - 273.15:8.3f} C | "
        f"Tavg = {np.mean(T) - 273.15:8.3f} C | "
        f"sigma = {sigma_now / 1e6:7.2f} MS/m | "
        f"max dT = {max_change:.3e} K | "
        f"wall = {now - last_print:.2f} s")


    last_print = now


if n > MIN_STEPS_BEFORE_STEADY_CHECK and max_change < STEADY_TOL_K:
```

```
            print(f"{case_name} reached near steady state at "
                f"t = {n * DT:.1f} s, max dT = {max_change:.3e} K")
            break

    dvol = geom["dx"] * geom["dy"] * geom["dz"]

    return {
        "case_name": case_name,
        "current_A": current_A,
        "T": T,
        "V": V,
        "Jx": Jx,
        "Jy": Jy,
        "Jz": Jz,
        "q_joule": q_joule,
        "q_cell": q_cell,
        "times": np.array(times),
        "Tmax_hist": np.array(Tmax_hist),
        "Tavg_hist": np.array(Tavg_hist),
        "Tmax_final": float(np.max(T)),
        "Tavg_final": float(np.mean(T)),
        "joule_W": float(np.sum(q_joule) * dvol),
        "cell_W": float(np.sum(q_cell) * dvol),
        "sigma_final": float(sigma_now),}


def passes_ampacity(result):
    return (
        result["Tmax_final"] <= T_MAX_LOCAL_K
```

```
        and result["Tavg_final"] <= T_MAX_AVG_K)

"""***Run 2***"""

def run_current_sweep():
    geom = build_geometry()

    print("\nGeometry")
    print("--------")
    print(f"Grid: {NX} x {NY} x {NZ} = {NX * NY * NZ:,} nodes")
    print(f"Cell diameter: {CELL_D * 1000:.1f} mm")
    print(f"Closest-packed row pitch: {ROW_Y_PITCH * 1000:.2f} mm")
    print(f"Busbar length: {BUSBAR_L * 1000:.1f} mm")
    print(f"Busbar width: {BUSBAR_W * 1000:.1f} mm")
    print(f"Busbar width in cell diameters: {BUSBAR_W / CELL_D:.3f}D")
    print(f"Busbar thick: {BUSBAR_T * 1000:.1f} mm")
    print(f"Contact diameter: {CONTACT_DIAMETER * 1000:.1f} mm")
    print(f"Cell internal R: {CELL_INTERNAL_R * 1000:.3f} mOhm/cell")
    print(f"Weld/contact R: {WELD_R * 1000:.3f} mOhm/contact")

    if N_HEAT_PIPES == 1:
        print("Heat pipes: 1 centered strip")
    elif N_HEAT_PIPES == 2:
        print("Heat pipes: 2 side strips")
    else:
        print("Heat pipes: N/A")

    print(f"Heat pipe width: {HP_WIDTH * 1000:.1f} mm")
    print(f"Heat pipe height: {HP_HEIGHT * 1000:.1f} mm")
```

```
print(f"Heat pipe electrical relative conductivity: {HP_ELECTRICAL_RELATIVE:.3
    f}")
print(f"Current starts at {START_CURRENT_A:.1f} A and steps by {CURRENT_STEP_A
    :.1f} A")

plot_geometry(geom)

ebase_solid = solve_base_electrical_1A(geom, "solid")
ebase_hp = solve_base_electrical_1A(geom, "heat_pipe")

thermal_lu_solid = build_thermal_lu(geom, "solid")
thermal_lu_hp = build_thermal_lu(geom, "heat_pipe")

sweep = []

current = START_CURRENT_A

final_solid = None
final_hp = None

while current <= MAX_CURRENT_A:
    print("\n" + "#" * 80)
    print(f"CURRENT CASE: {current:.1f} A")
    print("#" * 80)

    I_cell = current / N_CELLS_TOTAL
    P_cell_each = I_cell ** 2 * CELL_INTERNAL_R
    P_weld_each = I_cell ** 2 * WELD_R
```

```
print(f"Current per cell: {I_cell:.3f} A/cell")
print(f"Cell internal heat: {P_cell_each:.6f} W/cell")
print(f"Weld/contact heat: {P_weld_each:.6f} W/contact")
print(f"Total cell internal heat: {P_cell_each * N_CELLS_TOTAL:.6f} W")
print(f"Total weld/contact heat: {P_weld_each * N_CELLS_TOTAL:.6f} W")

res_solid = run_case(
    geom,
    ebase_solid,
    thermal_lu_solid,
    current,
    "solid")

res_hp = run_case(
    geom,
    ebase_hp,
    thermal_lu_hp,
    current,
    "heat_pipe")

solid_Tmax_C = res_solid["Tmax_final"] - 273.15
solid_Tavg_C = res_solid["Tavg_final"] - 273.15

hp_Tmax_C = res_hp["Tmax_final"] - 273.15
hp_Tavg_C = res_hp["Tavg_final"] - 273.15

sweep.append({
    "current_A": current,
    "solid_Tmax_C": solid_Tmax_C,
```

```
    "solid_Tavg_C": solid_Tavg_C,
    "hp_Tmax_C": hp_Tmax_C,
    "hp_Tavg_C": hp_Tavg_C,
    "solid": res_solid,
    "heat_pipe": res_hp,})

final_solid = res_solid
final_hp = res_hp

print("\nSummary")
print("-------")
print(
    f"Solid:  Tmax = {solid_Tmax_C:.3f} C, "
    f"Tavg = {solid_Tavg_C:.3f} C, "
    f"busbar Joule = {res_solid['joule_W']:.6f} W, "
    f"cell/weld = {res_solid['cell_W']:.6f} W, "
    f"passes = {passes_ampacity(res_solid)}")

print(
    f"HeatPipe: Tmax = {hp_Tmax_C:.3f} C, "
    f"Tavg = {hp_Tavg_C:.3f} C, "
    f"busbar Joule = {res_hp['joule_W']:.6f} W, "
    f"cell/weld = {res_hp['cell_W']:.6f} W, "
    f"passes = {passes_ampacity(res_hp)}")

if not passes_ampacity(res_solid) or not passes_ampacity(res_hp):
    print("\nStopping ampacity input maxed")
    break
```

```
        current += CURRENT_STEP_A

    plot_sweep(sweep)

    print("\nPlotting final current case")
    final_current = sweep[-1]["current_A"]
    print(f"Final current = {final_current:.1f} A")

    plot_temperature(geom, final_solid)
    plot_temperature(geom, final_hp)

    plot_joule(geom, final_solid)
    plot_joule(geom, final_hp)

    plot_current_density(geom, final_solid)
    plot_current_density(geom, final_hp)

    plot_history(final_solid, final_hp)

    return sweep

"""***Plotting***"""

def heat_pipe_label():
    if N_HEAT_PIPES == 1:
        return "center heat pipe"
    elif N_HEAT_PIPES == 2:
        return "two side heat pipes"
    else:
```

```
        return "heat pipe"


def plot_geometry(geom):
    x = geom["x"] * 1000
    y = geom["y"] * 1000

    plt.figure(figsize=(12, 5))
    ax = plt.gca()
    fig = plt.gcf()

    fig.patch.set_facecolor("black")
    ax.set_facecolor("black")

    busbar_background = np.ones((NX, NY))
    plt.contourf(x, y, busbar_background.T, levels=4, colors=["#111111"])

    for cx, cy, row_name in geom["centers"]:
        cell_outline = plt.Circle(
            (cx * 1000, cy * 1000),
            CELL_R * 1000,
            fill=False,
            color="white",
            linewidth=0.9)

        contact_outline = plt.Circle(
            (cx * 1000, cy * 1000),
            CONTACT_R * 1000,
            fill=False,
```

```
        color="red",
        linewidth=1.2)

    ax.add_patch(cell_outline)
    ax.add_patch(contact_outline)

plt.contour(
    x,
    y,
    geom["output_mask"][:, :, 0].T.astype(float),
    levels=[0.5],
    colors="yellow",
    linewidths=2)

hp_top_view = np.any(geom["hp_mask"], axis=2)

plt.contour(
    x,
    y,
    hp_top_view.T.astype(float),
    levels=[0.5],
    colors="cyan",
    linewidths=1.8)

plt.xlabel("x [mm]", color="white")
plt.ylabel("y [mm]", color="white")

ax.tick_params(colors="white")
```

```
    for spine in ax.spines.values():
        spine.set_color("white")

    ax.set_aspect("equal", adjustable="box")
    plt.tight_layout()
    plt.show()


def plot_temperature(geom, result):
    x = geom["x"] * 1000
    y = geom["y"] * 1000

    T_C = np.mean(result["T"], axis=2) - 273.15

    plt.figure(figsize=(12, 4.8))
    plt.contourf(x, y, T_C.T, levels=40)
    plt.colorbar(label="Averaged temperature [C]")

    ax = plt.gca()

    for cx, cy, row_name in geom["centers"]:
        cell_outline = plt.Circle(
            (cx * 1000, cy * 1000),
            CELL_R * 1000,
            fill=False,
            color="white",
            linewidth=0.8)

        ax.add_patch(cell_outline)
```

```
    plt.contour(
        x,
        y,
        geom["output_mask"][:, :, 0].T.astype(float),
        levels=[0.5],
        colors="yellow",
        linewidths=2)

    hp_top_view = np.any(geom["hp_mask"], axis=2)

    plt.contour(
        x,
        y,
        hp_top_view.T.astype(float),
        levels=[0.5],
        colors="cyan",
        linewidths=1.2)

    plt.xlabel("x [mm]")
    plt.ylabel("y [mm]")

    ax.set_aspect("equal", adjustable="box")
    plt.tight_layout()
    plt.show()


def plot_joule(geom, result):
    x = geom["x"] * 1000
```

```
y = geom["y"] * 1000

q_avg = np.mean(result["q_joule"], axis=2)

plt.figure(figsize=(12, 4.8))
plt.contourf(x, y, q_avg.T, levels=40)
plt.colorbar(label="Averaged Joule heating [W/m^3]")

ax = plt.gca()

for cx, cy, row_name in geom["centers"]:
    cell_outline = plt.Circle(
        (cx * 1000, cy * 1000),
        CELL_R * 1000,
        fill=False,
        color="white",
        linewidth=0.8)

    ax.add_patch(cell_outline)

plt.contour(
    x,
    y,
    geom["output_mask"][:, :, 0].T.astype(float),
    levels=[0.5],
    colors="yellow",
    linewidths=2)

hp_top_view = np.any(geom["hp_mask"], axis=2)
```

```
    plt.contour(
        x,
        y,
        hp_top_view.T.astype(float),
        levels=[0.5],
        colors="cyan",
        linewidths=1.2)

    plt.xlabel("x [mm]")
    plt.ylabel("y [mm]")

    ax.set_aspect("equal", adjustable="box")
    plt.tight_layout()
    plt.show()


def plot_current_density(geom, result):
    x = geom["x"] * 1000
    y = geom["y"] * 1000

    Jmag = np.sqrt(result["Jx"] ** 2 + result["Jy"] ** 2 + result["Jz"] ** 2)

    Jmag_avg = np.mean(Jmag, axis=2)
    Jx_avg = np.mean(result["Jx"], axis=2)
    Jy_avg = np.mean(result["Jy"], axis=2)

    plt.figure(figsize=(12, 4.8))
    plt.contourf(x, y, Jmag_avg.T, levels=40)
```

```
plt.colorbar(label="thickness-averaged current density [A/m^2]")

skip = 3

plt.streamplot(
    x[::skip],
    y[::skip],
    Jx_avg[::skip, ::skip].T,
    Jy_avg[::skip, ::skip].T,
    density=1.0,
    linewidth=0.7)

ax = plt.gca()

for cx, cy, row_name in geom["centers"]:
    cell_outline = plt.Circle(
        (cx * 1000, cy * 1000),
        CELL_R * 1000,
        fill=False,
        color="white",
        linewidth=0.8)

    ax.add_patch(cell_outline)

plt.contour(
    x,
    y,
    geom["output_mask"][:, :, 0].T.astype(float),
    levels=[0.5],
```

```
        colors="yellow",
        linewidths=2)

    hp_top_view = np.any(geom["hp_mask"], axis=2)

    plt.contour(
        x,
        y,
        hp_top_view.T.astype(float),
        levels=[0.5],
        colors="cyan",
        linewidths=1.2)

    plt.xlabel("x [mm]")
    plt.ylabel("y [mm]")

    ax.set_aspect("equal", adjustable="box")
    plt.tight_layout()
    plt.show()


def plot_history(res_solid, res_hp):
    plt.figure(figsize=(8, 5))

    plt.plot(res_solid["times"], res_solid["Tmax_hist"] - 273.15, label="solid max
         temp")
    plt.plot(res_solid["times"], res_solid["Tavg_hist"] - 273.15, label="solid
        average temp")
```

```
    plt.plot(res_hp["times"], res_hp["Tmax_hist"] - 273.15, label=f"{
        heat_pipe_label()} max temp")
    plt.plot(res_hp["times"], res_hp["Tavg_hist"] - 273.15, label=f"{
        heat_pipe_label()} average temp")

    plt.axhline(T_MAX_LOCAL_K - 273.15, linestyle="--", label="80 C max limit")
    plt.axhline(T_MAX_AVG_K - 273.15, linestyle="--", label="60 C average limit")

    plt.xlabel("time [s]")
    plt.ylabel("temperature [C]")

    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()


def plot_sweep(sweep):
    currents = np.array([s["current_A"] for s in sweep])
    solid_tmax = np.array([s["solid_Tmax_C"] for s in sweep])
    hp_tmax = np.array([s["hp_Tmax_C"] for s in sweep])

    plt.figure(figsize=(8, 5))

    plt.plot(currents, solid_tmax, marker="o", label="solid max temp")
    plt.plot(currents, hp_tmax, marker="o", label=f"{heat_pipe_label()} max temp")

    plt.axhline(T_MAX_LOCAL_K - 273.15, linestyle="--", label="80 C max limit")
    plt.axhline(T_MAX_AVG_K - 273.15, linestyle="--", label="60 C average limit")
```

```
    plt.xlabel("total current [A]")
    plt.ylabel("maximum temperature [C]")

    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

"""***Sensitivity Analysis General***"""

def run_one_sensitivity_case(current_A, case_name):
    geom = build_geometry()

    ebase = solve_base_electrical_1A(geom, case_name)
    thermal_lu = build_thermal_lu(geom, case_name)

    result = run_case(
        geom,
        ebase,
        thermal_lu,
        current_A,
        case_name)

    return result


def make_sensitivity_row(test_name, test_value, result):
    return {
```

```
        "test": test_name,
        "value": test_value,
        "current_A": result["current_A"],
        "Tmax_C": result["Tmax_final"] - 273.15,
        "Tavg_C": result["Tavg_final"] - 273.15,
        "joule_W": result["joule_W"],
        "cell_W": result["cell_W"],}


def print_sensitivity_table(rows):
    print("\nSensitivity Results")
    print("--------------------")
    print(f"{'test':<28s} {'value':>12s} {'Tmax [C]':>12s} {'Tavg [C]':>12s}")

    for row in rows:
        print(
            f"{row['test']:<28s} "
            f"{row['value']:>12.4g} "
            f"{row['Tmax_C']:>12.3f} "
            f"{row['Tavg_C']:>12.3f}")


def plot_sensitivity(rows, xlabel_text, title_text):
    values = [row["value"] for row in rows]
    Tmax = [row["Tmax_C"] for row in rows]
    Tavg = [row["Tavg_C"] for row in rows]

    plt.figure(figsize=(7, 5))
    plt.plot(values, Tmax, marker="o", label="Maximum temperature")
```

```
    plt.plot(values, Tavg, marker="s", label="Average temperature")

    plt.xlabel(xlabel_text)
    plt.ylabel("Temperature [C]")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

"""***Sensitivity Analysis Cell Heat Genaration***"""

def sensitivity_cell_heat_rate(current_A=350.0):
    global CELL_FIXED_HEAT_W

    original_cell_fixed_heat = CELL_FIXED_HEAT_W

    cell_heat_rates = [
        0.1,
        0.25,
        0.50,
        1.00,
        2.00]

    rows = []

    for cell_heat in cell_heat_rates:
        print("\n" + "=" * 80)
        print(f"Testing added cell heat rate = {cell_heat:.2f} W/cell")
        print("=" * 80)
```

```
        CELL_FIXED_HEAT_W = cell_heat

        result = run_one_sensitivity_case(
            current_A,
            "heat_pipe")

        rows.append(make_sensitivity_row(
            "Cell heat [W/cell]",
            cell_heat,
            result))

    CELL_FIXED_HEAT_W = original_cell_fixed_heat

    print_sensitivity_table(rows)

    plot_sensitivity(
        rows,
        "Added heat generation per cell [W/cell]",
        "Sensitivity to cell heat generation")

    return rows


cell_heat_sensitivity = sensitivity_cell_heat_rate(current_A=350.0)

"""***Sensitivity Analysis Heat Pipe Spacing From Edge***"""

def sensitivity_heat_pipe_edge_spacing(current_A=350.0):
```

```
global HP_EDGE_OFFSET

original_edge_offset = HP_EDGE_OFFSET

edge_offsets = [
    4.0e-3,
    8.0e-3,
    12.0e-3,
    16.0e-3,
    20.0e-3,
    24.0e-3]

rows = []

for edge_offset in edge_offsets:
    print("\n" + "=" * 80)
    print(f"Testing heat pipe edge spacing = {edge_offset * 1000:.2f} mm")
    print("=" * 80)

    HP_EDGE_OFFSET = edge_offset

    result = run_one_sensitivity_case(
        current_A,
        "heat_pipe")

    rows.append(make_sensitivity_row(
        "HP edge spacing [m]",
        edge_offset,
        result))
```

```
    HP_EDGE_OFFSET = original_edge_offset

    print_sensitivity_table(rows)

    plot_sensitivity(
        rows,
        "Heat pipe distance from edge [m]",
        "Sensitivity to heat pipe edge spacing")

    return rows


spacing_sensitivity = sensitivity_heat_pipe_edge_spacing(current_A=350.0)

"""***Sensitivity Analysis Heat Pipe Thermal Conductivity***"""

def sensitivity_heat_pipe_thermal_conductivity(current_A=350.0):
    global K_HP_AXIAL

    original_k_hp_axial = K_HP_AXIAL

    k_values = [
        1000.0,
        5000.0,
        15000.0,
        30000.0,
        50000.0]
```

```
rows = []

for k_hp in k_values:
    print("\n" + "=" * 80)
    print(f"Testing heat pipe axial thermal conductivity = {k_hp:.1f} W/m-K")
    print("=" * 80)

    K_HP_AXIAL = k_hp

    result = run_one_sensitivity_case(
        current_A,
        "heat_pipe")

    rows.append(make_sensitivity_row(
        "HP thermal k [W/m-K]",
        k_hp,
        result))

K_HP_AXIAL = original_k_hp_axial

print_sensitivity_table(rows)

plot_sensitivity(
    rows,
    "Heat pipe axial thermal conductivity [W/m-K]",
    "Sensitivity to heat pipe thermal conductivity")

return rows
```

```
hp_thermal_sensitivity = sensitivity_heat_pipe_thermal_conductivity(current_A
    =350.0)


"""***Applying Sensitivity Analysis***"""


def get_best_sensitivity_value(rows):
    best_row = rows[0]

    for row in rows:
        if row["Tmax_C"] < best_row["Tmax_C"]:
            best_row = row

    return best_row



# Find the heat pipe spacing case with the lowest maximum temperature
best_spacing = get_best_sensitivity_value(spacing_sensitivity)



def apply_best_sensitivity_choices():
    global HP_EDGE_OFFSET

    HP_EDGE_OFFSET = best_spacing["value"]

    print("\nUpdated model constants")
    print("=======================")
    print(f"HP_EDGE_OFFSET = {HP_EDGE_OFFSET}")
```

```
apply_best_sensitivity_choices()

"""***Convergence Study***"""

def run_actual_model_convergence_study(current_A=350.0, case_name="heat_pipe"):

    global DT

    original_dt = DT
    baseline_dt = 2.0

    # These are centered around the final value used in the report.
    dt_values = [1.0, 1.5, 2.0, 3.0, 4.0]

    rows = []

    print("\nActual Model Convergence Study")
    print("==============================")
    print(f"Case: {case_name}")
    print(f"Current: {current_A:.1f} A")
    print(f"Baseline dt: {baseline_dt:.2f} s")

    for dt in dt_values:
        print("\n" + "-" * 70)
        print(f"Testing dt = {dt:.2f} s")
        print("-" * 70)

        DT = dt
```

```
    geom = build_geometry()
    ebase = solve_base_electrical_1A(geom, case_name)
    thermal_lu = build_thermal_lu(geom, case_name)

    result = run_case(
        geom,
        ebase,
        thermal_lu,
        current_A,
        case_name)

    rows.append({
        "dt_s": dt,
        "dt_change_percent": (dt - baseline_dt) / baseline_dt * 100.0,
        "Tmax_C": result["Tmax_final"] - 273.15,
        "Tavg_C": result["Tavg_final"] - 273.15})

# Restore original value so the rest of the report still uses the chosen dt.
DT = original_dt

baseline_row = None
for row in rows:
    if abs(row["dt_s"] - baseline_dt) < 1e-12:
        baseline_row = row

if baseline_row is None:
    raise ValueError("Baseline dt was not included in dt_values.")
```

```
baseline_Tmax = baseline_row["Tmax_C"]


for row in rows:
    row["Tmax_change_percent"] = (
        (row["Tmax_C"] - baseline_Tmax) / baseline_Tmax * 100.0)


print("\nConvergence Study Results")
print("-------------------------")
print(
    f"{'dt [s]':>10s} "
    f"{'dt change [%]':>16s} "
    f"{'Tmax [C]':>12s} "
    f"{'Tavg [C]':>12s} "
    f"{'Tmax change [%]':>18s}")


for row in rows:
    print(
        f"{row['dt_s']:10.3f} "
        f"{row['dt_change_percent']:16.3f} "
        f"{row['Tmax_C']:12.3f} "
        f"{row['Tavg_C']:12.3f} "
        f"{row['Tmax_change_percent']:18.6f}")


x = [row["dt_change_percent"] for row in rows]
y = [row["Tmax_change_percent"] for row in rows]


plt.figure(figsize=(7, 5))
plt.plot(x, y, marker="o")
plt.axhline(0.0, linestyle="--")
```

```
    plt.axvline(0.0, linestyle="--")
    plt.xlabel("Change in time step, $\\Delta t$ [%]")
    plt.ylabel("Change in final maximum temperature [%]")
    plt.grid(True)
    plt.tight_layout()
    plt.savefig("actual_model_convergence_study.png", dpi=300)
    plt.show()

    return rows


actual_convergence_rows = run_actual_model_convergence_study(
    current_A=350.0,
    case_name="heat_pipe")

"""***Verification code***"""

def exact_cooling_solution(t, T0, T_inf, a):
    return T_inf + (T0 - T_inf)*np.exp(-a*t)


def backward_euler_cooling(dt, t_final, T0, T_inf, a):
    n_steps = int(t_final / dt)

    t = np.zeros(n_steps + 1)
    T = np.zeros(n_steps + 1)

    T[0] = T0
```

```
    for n in range(n_steps):
        t[n + 1] = t[n] + dt

        # Backward Euler update:
        # T_next = T_old - dt*a*(T_next - T_inf)
        # Solve for T_next:
        T[n + 1] = (T[n] + dt*a*T_inf) / (1.0 + dt*a)

    return t, T


def run_verification():
    T0 = 80.0        # deg C
    T_inf = 25.0    # deg C
    a = 0.015       # 1/s
    t_final = 300.0 # s

    dt_values = [10.0, 5.0, 2.5, 1.25, 0.625]

    errors = []

    plt.figure(figsize=(7, 5))

    for dt in dt_values:
        t, T_num = backward_euler_cooling(dt, t_final, T0, T_inf, a)
        T_exact = exact_cooling_solution(t, T0, T_inf, a)

        final_error = abs(T_num[-1] - T_exact[-1])
        percent_error = final_error / abs(T_exact[-1]) * 100.0
```

```
    errors.append(percent_error)

    plt.plot(t, T_num, marker="o", markersize=3, label=f"dt = {dt:g} s")

t_exact = np.linspace(0.0, t_final, 500)
T_exact_smooth = exact_cooling_solution(t_exact, T0, T_inf, a)

plt.plot(t_exact, T_exact_smooth, linewidth=2, label="Exact solution")

plt.xlabel("Time [s]")
plt.ylabel("Temperature [C]")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("verification_temperature_plot.png", dpi=300)
plt.show()

plt.figure(figsize=(7, 5))
plt.loglog(dt_values, errors, marker="o")
plt.xlabel("Time step, $\\Delta t$ [s]")
plt.ylabel("Final temperature error [%]")
plt.grid(True, which="both")
plt.tight_layout()
plt.savefig("verification_error_plot.png", dpi=300)
plt.show()

print("\nVerification Results")
print("--------------------")
```

```
    print(f"{'dt [s]':>10s} {'Percent error [%]':>20s}")

    for dt, error in zip(dt_values, errors):
        print(f"{dt:10.4f} {error:20.6f}")

    return dt_values, errors


verification_dt, verification_errors = run_verification()

"""***Actual Running Code***"""

sweep_results = run_current_sweep()
```